III Techniques for Better Deep Learning Models

Save and Load Your PyTorch Models

13

A deep learning model is a mathematical abstraction of data, in which a lot of parameters are involved. Training these parameters can take hours, days, and even weeks but afterward, you can make use of the result to apply on new data. This is called inference in machine learning. It is important to know how we can preserve the trained model in disk and later, load it for use in inference. In this chapter, you will discover how to save your PyTorch models to files and load them up again to make predictions. After reading this chapter, you will know:

- ▷ What are states and parameters in a PyTorch model
- ▷ How to save model states
- ▷ How to load model states

Let's get started.

Overview

This chapter is in three parts; they are

- ▷ Build an Example Model
- ▷ What's Inside a PyTorch Model
- ▷ Accessing `state_dict` of a Model

13.1 Build an Example Model

Let's start with a very simple model in PyTorch. It is a model based on the iris dataset (see Chapter 10). You will load the dataset using scikit-learn (which the targets are integer labels 0, 1, and 2) and train a neural network for this multiclass classification problem. In this model, you used log softmax as the output activation so you can combine with the negative log likelihood loss function. It is equivalent to no output activation combined with cross-entropy loss function.

```
import torch
import torch.nn as nn
import torch.optim as optim
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

# Load data into NumPy arrays
data = load_iris()
X, y = data["data"], data["target"]

# convert NumPy array into PyTorch tensors
X = torch.tensor(X, dtype=torch.float32)
y = torch.tensor(y, dtype=torch.long)

# split
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size=0.7, shuffle=True)

# PyTorch model
class Multiclass(nn.Module):
    def __init__(self):
        super().__init__()
        self.hidden = nn.Linear(4, 8)
        self.act = nn.ReLU()
        self.output = nn.Linear(8, 3)
        self.logsoftmax = nn.LogSoftmax(dim=1)

    def forward(self, x):
        x = self.act(self.hidden(x))
        x = self.logsoftmax(self.output(x))
        return x

model = Multiclass()

# loss metric and optimizer
loss_fn = nn.NLLLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

# prepare model and training parameters
n_epochs = 100
batch_size = 5
batch_start = torch.arange(0, len(X), batch_size)

# training loop
for epoch in range(n_epochs):
    for start in batch_start:
        # take a batch
        X_batch = X_train[start:start+batch_size]
        y_batch = y_train[start:start+batch_size]
        # forward pass
        y_pred = model(X_batch)
        loss = loss_fn(y_pred, y_batch)
        # backward pass
        optimizer.zero_grad()
        loss.backward()
```

```
        # update weights
        optimizer.step()
```

Listing 13.1: An example model trained with negative log-likelihood loss function

With such a simple model and small dataset, it shouldn't take a long time to finish training. Afterwards, we can confirm that this model works, by evaluating it with the test set:

```
...
y_pred = model(X_test)
acc = (torch.argmax(y_pred, 1) == y_test).float().mean()
print("Accuracy: %.2f" % acc)
```

Listing 13.2: Evaluating a model with test set

It prints, for example,

```
Accuracy: 0.96
```

Output 13.1: Accuracy as evaluated on the test set

13.2 What's Inside a PyTorch Model

PyTorch model is an object in Python. It holds some deep learning building blocks such as various kinds of layers and activation functions. It also knows how to connect them so it can produce you an output from your input tensors. The algorithm of a model is fixed at the time you created it, however, it has trainable parameters that is supposed to be modified during training loop so the model can be more accurate.

You saw how to get the model parameters when you setup the optimizer for your training loop, namely,

```
optimizer = optim.Adam(model.parameters(), lr=0.001)
```

Listing 13.3: Creating an optimizer

The function `model.parameters()` give you a generator that reference to each layers' trainable parameters in turn in the form of PyTorch tensors. Therefore, it is possible for you to make a copy of them or overwrite them, for example:

```
# create a new model
newmodel = Multiclass()
# ask PyTorch to ignore autograd on update and overwrite parameters
with torch.no_grad():
    for newtensor, oldtensor in zip(newmodel.parameters(), model.parameters()):
        newtensor.copy_(oldtensor)
# test with new model using copied tensor
y_pred = newmodel(X_test)
acc = (torch.argmax(y_pred, 1) == y_test).float().mean()
print("Accuracy: %.2f" % acc)
```

Listing 13.4: Copying weights from one model to another

Which the result should be exactly the same as before since you essentially made the two models identical by copying the parameters.

However, this is not always the case. Some models has *non-trainable parameters*. One example is the batch normalization layer that is common in many convolution neural networks. What it does is to apply normalization on tensors that produced by its previous layer and pass on the normalized tensor to its next layer. It has two parameters: The mean and standard deviation, which are learned from your input data during training loop but not trainable by the optimizer. Therefore these are not part of `model.parameters()` but equally important.

13.3 Accessing `state_dict` of a Model

To access all parameters of a model, trainable or not, you can get it from `state_dict()` function. From the model above, this is what you can get:

```
import pprint
pp = pprint.PrettyPrinter(indent=4)
pp.pprint(model.state_dict())
```

Listing 13.5: Pretty-print all parameters from a model

The model above produces the following:

```
OrderedDict([  (  'hidden.weight',
                   tensor([[ 0.1480,  0.0336,  0.3425,  0.2832],
        [ 0.5265,  0.8587, -0.7023, -1.1149],
        [ 0.1620,  0.8440, -0.6189, -0.6513],
        [-0.1559,  0.0393, -0.4701,  0.0825],
        [ 0.6364, -0.6622,  1.1150,  0.9162],
        [ 0.2081, -0.0958, -0.2601, -0.3148],
        [-0.0804,  0.1027,  0.7363,  0.6068],
        [-0.4101, -0.3774, -0.1852,  0.1524]])),
               (  'hidden.bias',
                  tensor([ 0.2057,  0.7998, -0.0578,  0.1041, -0.3903, -0.4521, -0.5307, -0.1532])),
               (  'output.weight',
                  tensor([[-0.0954,  0.8683,  1.0667,  0.2382, -0.4245, -0.0409, -0.2587, -0.0745],
        [-0.0829,  0.8642, -1.6892, -0.0188,  0.0420, -0.1020,  0.0344, -0.1210],
        [-0.0176, -1.2809, -0.3040,  0.1985,  0.2423,  0.3333,  0.4523, -0.1928]])),
               ('output.bias', tensor([ 0.0998,  0.6360, -0.2990]))])
```

Output 13.2: The `state_dict` from a model

It is called `state_dict` because all state variables of a model are here. It is an `OrderedDict` object from Python's built-in `collections` module. All components from a PyTorch model has a name and so as the parameters therein. The `OrderedDict` object allows you to map the weights back to the parameters correctly by matching their names.

This is how you should save and load the model: Fetch the model states into an `OrderedDict`, serialize and save it to disk. For inference, you create a model first (without training), and load the states. In Python, the native format for serialization is pickle:

```
import pickle

# Save model
with open("iris-model.pickle", "wb") as fp:
    pickle.dump(model.state_dict(), fp)

# Create new model and load states
newmodel = Multiclass()
with open("iris-model.pickle", "rb") as fp:
    newmodel.load_state_dict(pickle.load(fp))

# test with new model using copied tensor
y_pred = newmodel(X_test)
acc = (torch.argmax(y_pred, 1) == y_test).float().mean()
print("Accuracy: %.2f" % acc)
```

Listing 13.6: Serialize model parameters using pickle

You know it works because the model you didn't train produced the same result as the one you trained.

Indeed, the recommended way is to use the PyTorch API to save and load the states, instead of using pickle manually:

```
# Save model
torch.save(model.state_dict(), "iris-model.pth")

# Create new model and load states
newmodel = Multiclass()
newmodel.load_state_dict(torch.load("iris-model.pth"))

# test with new model using copied tensor
y_pred = newmodel(X_test)
acc = (torch.argmax(y_pred, 1) == y_test).float().mean()
print("Accuracy: %.2f" % acc)
```

Listing 13.7: Serialize model parameters using PyTorch save and load functions

The `*.pth` file is indeed a zip file of some pickle files created by PyTorch. It is recommended because PyTorch can store additional information in it. Note that you stored only the states but not the model. You still need to create the model using Python code and load the states into it. If you wish to store the model as well, you can pass in the entire model instead of the states:

```
# Save model
torch.save(model, "iris-model-full.pth")

# Load model
newmodel = torch.load("iris-model-full.pth")

# test with new model using copied tensor
y_pred = newmodel(X_test)
acc = (torch.argmax(y_pred, 1) == y_test).float().mean()
print("Accuracy: %.2f" % acc)
```

Listing 13.8: Saving and loading a PyTorch model

But remember, due to the nature of Python language, doing so does not relieve you from keeping the code of the model. The `newmodel` object above is an instance of `Multiclass` class that you defined before. When you load the model from disk, Python need to know in detail how this class is defined. If you run a script with just the line `torch.load()`, you will see the following error message:

```
Traceback (most recent call last):
File "<stdin>", line 1, in <module>
File "/.../torch/serialization.py", line 789, in load
return _load(opened_zipfile, map_location, pickle_module, **pickle_load_args)
File "/.../torch/serialization.py", line 1131, in _load
result = unpickler.load()
File "/.../torch/serialization.py", line 1124, in find_class
return super().find_class(mod_name, name)
AttributeError: Can't get attribute 'Multiclass' on <module '__main__' (built-in)>
```

Output 13.3: Error message for unable to find the code of a model class

That's why it is recommended to save only the state dict rather than the entire model.

Putting everything together, the following is the complete code to demonstrate how to create a model, train it, and save to disk:

```
import torch
import torch.nn as nn
import torch.optim as optim
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

# Load data into NumPy arrays
data = load_iris()
X, y = data["data"], data["target"]

# convert NumPy array into PyTorch tensors
X = torch.tensor(X, dtype=torch.float32)
y = torch.tensor(y, dtype=torch.long)

# split
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size=0.7, shuffle=True)

# PyTorch model
class Multiclass(nn.Module):
    def __init__(self):
        super().__init__()
        self.hidden = nn.Linear(4, 8)
        self.act = nn.ReLU()
        self.output = nn.Linear(8, 3)
        self.logsoftmax = nn.LogSoftmax(dim=1)

    def forward(self, x):
        x = self.act(self.hidden(x))
        x = self.logsoftmax(self.output(x))
        return x
```

```
model = Multiclass()

# loss metric and optimizer
loss_fn = nn.NLLLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

# prepare model and training parameters
n_epochs = 100
batch_size = 5
batch_start = torch.arange(0, len(X), batch_size)

# training loop
for epoch in range(n_epochs):
    for start in batch_start:
        # take a batch
        X_batch = X_train[start:start+batch_size]
        y_batch = y_train[start:start+batch_size]
        # forward pass
        y_pred = model(X_batch)
        loss = loss_fn(y_pred, y_batch)
        # backward pass
        optimizer.zero_grad()
        loss.backward()
        # update weights
        optimizer.step()

# Save model
torch.save(model.state_dict(), "iris-model.pth")
```

Listing 13.9: Create, train, and save a model

And the following is how to load the model from disk and run it for inference:

```
import torch
import torch.nn as nn
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

# Load data into NumPy arrays
data = load_iris()
X, y = data["data"], data["target"]

# convert NumPy array into PyTorch tensors
X = torch.tensor(X, dtype=torch.float32)
y = torch.tensor(y, dtype=torch.long)

# split
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size=0.7, shuffle=True)

# PyTorch model
class Multiclass(nn.Module):
    def __init__(self):
        super().__init__()
```

```
        self.hidden = nn.Linear(4, 8)
        self.act = nn.ReLU()
        self.output = nn.Linear(8, 3)
        self.logsoftmax = nn.LogSoftmax(dim=1)

    def forward(self, x):
        x = self.act(self.hidden(x))
        x = self.logsoftmax(self.output(x))
        return x

# Create new model and load states
model = Multiclass()
model.load_state_dict(torch.load("iris-model.pth"))

# Run model for inference
y_pred = model(X_test)
acc = (torch.argmax(y_pred, 1) == y_test).float().mean()
print("Accuracy: %.2f" % acc)
```

Listing 13.10: Load a saved model for inference

13.4 Further Readings

This section provides more resources on the topic if you are looking to go deeper.

Articles

Iris Data Set. UCI Machine Learning Repository.
https://archive.ics.uci.edu/ml/datasets/Iris

Load and return the iris dataset. scikit-learn documentation.
https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_iris.html

Saving and loading models. PyTorch tutorial.
https://pytorch.org/tutorials/beginner/saving_loading_models.html

NLLLoss. PyTorch API.
https://pytorch.org/docs/stable/generated/torch.nn.NLLLoss.html

LogSoftmax. PyTorch API.
https://pytorch.org/docs/stable/generated/torch.nn.LogSoftmax.html

13.5 Summary

In this chapter, you learned how to keep a copy of your trained PyTorch model in disk and how to reuse it. In particular, you learned

- ▷ What are parameters and states in a PyTorch model
- ▷ How to save all necessary states from a model to disk
- ▷ How to rebuild a working model from the saved states

In the next chapter, you will learn how to select activation functions for your model.

Using Activation Functions in Deep Learning Models

14

A deep learning model in its simplest form are layers of perceptrons connected in tandem. Without any activation functions, they are just matrix multiplications with limited power, regardless how many of them. Activation is the magic why neural network can be an approximation to a wide variety of nonlinear function. In PyTorch, there are many activation functions available for use in your deep learning models. In this chapter, you will see how the choice of activation functions can impact the model. Specifically,

- ▷ What are the common activation functions
- ▷ What are the nature of activation functions
- ▷ How the different activation functions impact the learning rate
- ▷ How the selection of activation function can solve the vanishing gradient problem

Let's get started.

Overview

This chapter is in three parts; they are

- ▷ A Toy Model of Binary Classification
- ▷ Why Nonlinear Functions?
- ▷ The Effect of Activation Functions

14.1 A Toy Model of Binary Classification

Let's start with a simple example of binary classification. Here you use the `make_circle()` function from scikit-learn to create a synthetic dataset for binary classification. This dataset has two features: The x- and y-coordinate of points. Each point belongs to one of the two classes. You can generate 1000 data points and visualize them as below:

```
from sklearn.datasets import make_circles
import matplotlib.pyplot as plt
import torch

# Make data: Two circles on x-y plane as a classification problem
X, y = make_circles(n_samples=1000, factor=0.5, noise=0.1)
X = torch.tensor(X, dtype=torch.float32)
y = torch.tensor(y.reshape(-1, 1), dtype=torch.float32)

plt.figure(figsize=(8,6))
plt.scatter(X[:,0], X[:,1], c=y)
plt.show()
```

Listing 14.1: Creating a synthetic dataset for binary classification

The dataset is visualized as follows:

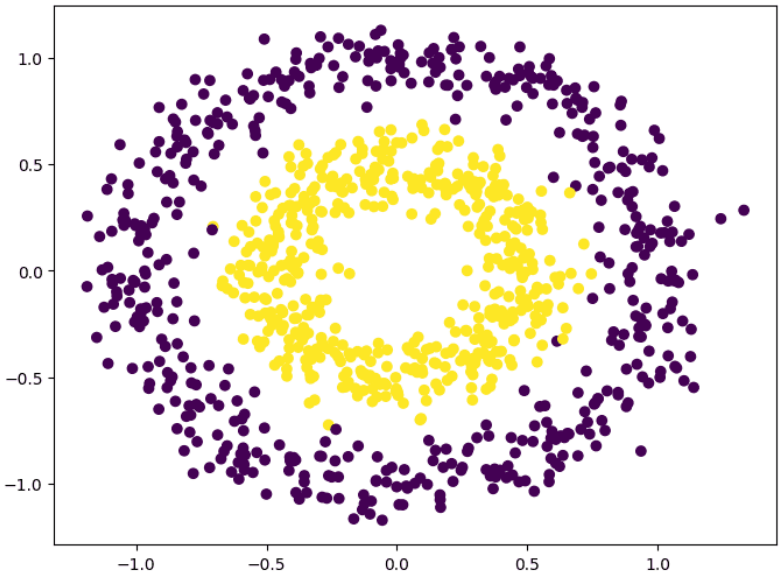


Figure 14.1: The synthetic dataset

This dataset is special because it is simple but not linearly separable: It is impossible to find a straight line to separate two classes. How can your neural network figure out there's a circle boundary between the classes is a challenge.

Let's create a deep learning model for this problem. To make things simple, you do not do cross-validation. You may find the neural network overfit the data but it doesn't affect the discussion below. The model has 4 hidden layers and the output layer gives a sigmodal value (0 to 1) for binary classification. The model accepts a parameter at its constructor to specify what is the activation to use in the hidden layers. You implement the training loop in a function as you will run this for several times.

The implementation is as follows:

```
class Model(nn.Module):
    def __init__(self, activation=nn.ReLU):
        super().__init__()
        self.layer0 = nn.Linear(2,5)
        self.act0 = activation()
        self.layer1 = nn.Linear(5,5)
        self.act1 = activation()
        self.layer2 = nn.Linear(5,5)
        self.act2 = activation()
        self.layer3 = nn.Linear(5,5)
        self.act3 = activation()
        self.layer4 = nn.Linear(5,1)
        self.act4 = nn.Sigmoid()

    def forward(self, x):
        x = self.act0(self.layer0(x))
        x = self.act1(self.layer1(x))
        x = self.act2(self.layer2(x))
        x = self.act3(self.layer3(x))
        x = self.act4(self.layer4(x))
        return x

def train_loop(model, X, y, n_epochs=300, batch_size=32):
    loss_fn = nn.BCELoss()
    optimizer = optim.Adam(model.parameters(), lr=0.0001)
    batch_start = torch.arange(0, len(X), batch_size)

    bce_hist = []
    acc_hist = []

    for epoch in range(n_epochs):
        # train model with optimizer
        model.train()
        for start in batch_start:
            X_batch = X[start:start+batch_size]
            y_batch = y[start:start+batch_size]
            y_pred = model(X_batch)
            loss = loss_fn(y_pred, y_batch)
            optimizer.zero_grad()
            loss.backward()
            optimizer.step()
        # evaluate BCE and accuracy at end of each epoch
        model.eval()
        with torch.no_grad():
            y_pred = model(X)
            bce = float(loss_fn(y_pred, y))
            acc = float((y_pred.round() == y).float().mean())
        bce_hist.append(bce)
        acc_hist.append(acc)
        # print metrics every 10 epochs
        if (epoch+1) % 10 == 0:
            print("Before epoch %d: BCE=%.4f, Accuracy=%.2f%%" % (epoch+1, bce, acc*100))
    return bce_hist, acc_hist
```

Listing 14.2: A PyTorch model for binary classification and a training loop function

At the end of each epoch in the training function, you evaluate the model with the entire dataset. The evaluation result is returned when the training finished. In the following, you create a model, train it, and plot the training history. The activation function you use is *rectified linear unit* or ReLU, which is the most common activation function nowadays:

```
activation = nn.ReLU
model = Model(activation=activation)
bce_hist, acc_hist = train_loop(model, X, y)
plt.plot(bce_hist, label="BCE")
plt.plot(acc_hist, label="Accuracy")
plt.xlabel("Epochs")
plt.ylim(0, 1)
plt.title(str(activation))
plt.legend()
plt.show()
```

Listing 14.3: Creating and training a model, with the training history visualized

Running this give you the following:

```
Before epoch 10: BCE=0.7025, Accuracy=50.00%
Before epoch 20: BCE=0.6990, Accuracy=50.00%
Before epoch 30: BCE=0.6959, Accuracy=50.00%
...
Before epoch 280: BCE=0.3051, Accuracy=96.30%
Before epoch 290: BCE=0.2785, Accuracy=96.90%
Before epoch 300: BCE=0.2543, Accuracy=97.00%
```

Output 14.1: Training history

and this plot:

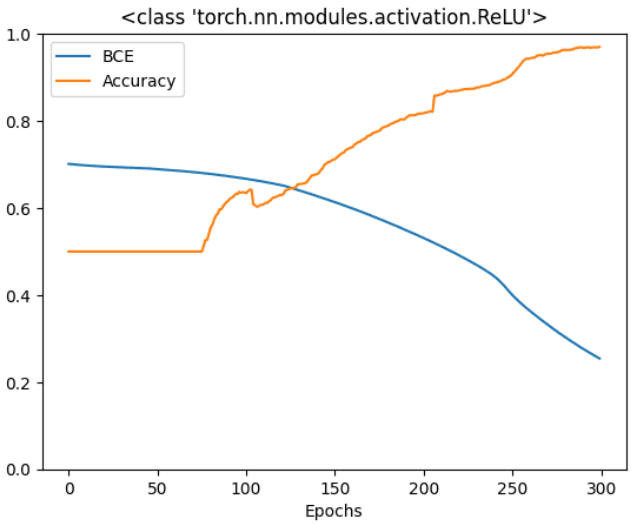


Figure 14.2: Training history visualized

This model works great. After 300 epochs, it can achieve 90% accuracy. However, ReLU is not the only activation function. Historically, sigmoid function and hyperbolic tangents were

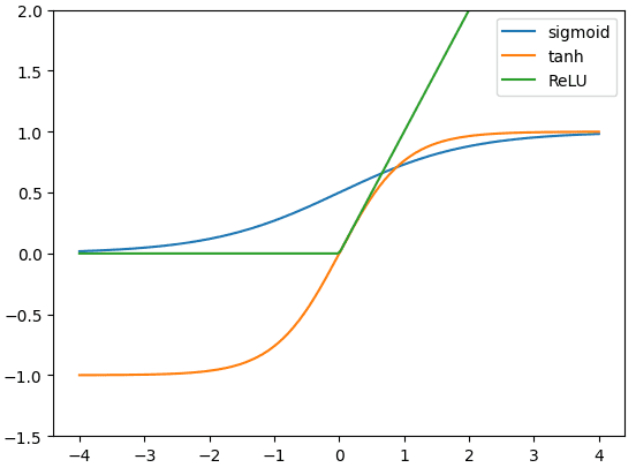
common in neural networks literature. If you're curious, below are how you can compare these three activation functions, using matplotlib:

```
import matplotlib.pyplot as plt
import torch
import torch.nn as nn

x = torch.linspace(-4, 4, 200)
relu = nn.ReLU()(x)
tanh = nn.Tanh()(x)
sigmoid = nn.Sigmoid()(x)

plt.plot(x, sigmoid, label="sigmoid")
plt.plot(x, tanh, label="tanh")
plt.plot(x, relu, label="ReLU")
plt.ylim(-1.5, 2)
plt.legend()
plt.show()
```

Listing 14.4: Comparing different activation functions



$$\mathrm{ReLU}(x) = \max(0, x)$$

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Figure 14.3: Plot and formula of different activation functions

ReLU is called rectified linear unit because it is a linear function $y = x$ at positive x but remains zero if x is negative. Mathematically, it is $y = \max(0, x)$. Hyperbolic tangent ($y = \tanh(x) = \dfrac{e^x - e^{-x}}{e^x + e^{-x}}$) goes from -1 to $+1$ smoothly while sigmoid function ($y = \sigma(x) = 1/(1 + e^{-x})$) goes from 0 to $+1$.

If you try to differentiate these functions, you will find that ReLU is the easiest: The gradient is 1 at positive region and 0 otherwise. Hyperbolic tangent has a steeper slope therefore its gradient is greater than that of sigmoid function.

All these functions are increasing. Therefore, their gradients are never negative. This is one of the criteria for an activation function suitable to use in neural networks.

14.2 Why Nonlinear Functions?

You might be wondering, why all this hype about nonlinear activation functions? Or why can't we just use an identity function after the weighted linear combination of activations from the previous layer? Using multiple linear layers is basically the same as using a single linear layer. This can be seen through a simple example. Let's say you have a one hidden layer neural network, each with two hidden neurons.

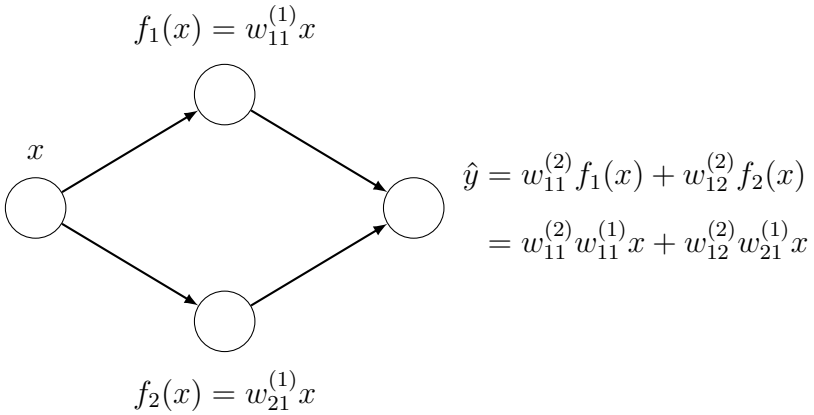


Figure 14.4: Single hidden layer neural network with linear layers

You can then rewrite the output layer as a linear combination of the original input variable if you used a linear hidden layer. If you had more neurons and weights, the equation would be a lot longer with more nesting and more multiplications between successive layer weights. However, the idea remains the same: You can represent the entire network as a single linear layer. To make the network represent more complex functions, you would need nonlinear activation functions.

14.3 The Effect of Activation Functions

To explain how much impact the activation function can bring to your model, let's modify the training loop function to capture more data: The gradients in each training step. You model has four hidden layers and one output layer. In each step, the backward pass calculates the gradient of the weights of each layer and the weight update is done by the optimizer based on the result of the backward pass. You should observe how the gradient changes as the training progressed. Therefore, the training loop function is modified to collect the mean absolute value of the gradient in each layer in each step, as follows:

```
def train_loop(model, X, y, n_epochs=300, batch_size=32):
    loss_fn = nn.BCELoss()
    optimizer = optim.Adam(model.parameters(), lr=0.0001)
    batch_start = torch.arange(0, len(X), batch_size)

    bce_hist = []
    acc_hist = []
    grad_hist = [[],[],[],[],[]]
```

```
    for epoch in range(n_epochs):
        # train model with optimizer
        model.train()
        layer_grad = [[],[],[],[],[]]
        for start in batch_start:
            X_batch = X[start:start+batch_size]
            y_batch = y[start:start+batch_size]
            y_pred = model(X_batch)
            loss = loss_fn(y_pred, y_batch)
            optimizer.zero_grad()
            loss.backward()
            optimizer.step()
            # collect mean absolute value of gradients
            layers = [model.layer0, model.layer1, model.layer2, model.layer3,
                      model.layer4]
            for n,layer in enumerate(layers):
                mean_grad = float(layer.weight.grad.abs().mean())
                layer_grad[n].append(mean_grad)
        # evaluate BCE and accuracy at end of each epoch
        model.eval()
        with torch.no_grad():
            y_pred = model(X)
            bce = float(loss_fn(y_pred, y))
            acc = float((y_pred.round() == y).float().mean())
        bce_hist.append(bce)
        acc_hist.append(acc)
        for n, grads in enumerate(layer_grad):
            grad_hist[n].append(sum(grads)/len(grads))
        # print metrics every 10 epochs
        if epoch % 10 == 9:
            print("Epoch %d: BCE=%.4f, Accuracy=%.2f%%" % (epoch, bce, acc*100))
    return bce_hist, acc_hist, layer_grad
```

Listing 14.5: Training loop function to collect statistics of the computed gradient

At the end of the inner for-loop, the gradients of the layer weights are computed by the backward process earlier and you can access to the gradient using `model.layer0.weight.grad`. Like the weights, the gradients are tensors. You take the absolute value of each element and then compute the mean over all elements. This value depends on the batch and can be very noisy. Thus you summarize all such mean absolute value over the same epoch at the end.

Note that you have five layers in the neural network (hidden and output layers combined). So you can see the pattern of each layer's gradient across the epochs if you visualize them. In below, you run the training loop as before and plot both the cross-entropy and accuracy as well as the mean absolute gradient of each layer:

```
activation = nn.ReLU
model = Model(activation=activation)
bce_hist, acc_hist, grad_hist = train_loop(model, X, y)

fig, ax = plt.subplots(1, 2, figsize=(12, 5))
ax[0].plot(bce_hist, label="BCE")
```

```
ax[0].plot(acc_hist, label="Accuracy")
ax[0].set_xlabel("Epochs")
ax[0].set_ylim(0, 1)
for n, grads in enumerate(grad_hist):
    ax[1].plot(grads, label="layer"+str(n))
ax[1].set_xlabel("Epochs")
fig.suptitle(str(activation))
ax[0].legend()
ax[1].legend()
plt.show()
```

Listing 14.6: Plotting the mean absolute gradient

Running the above produces the following plot:

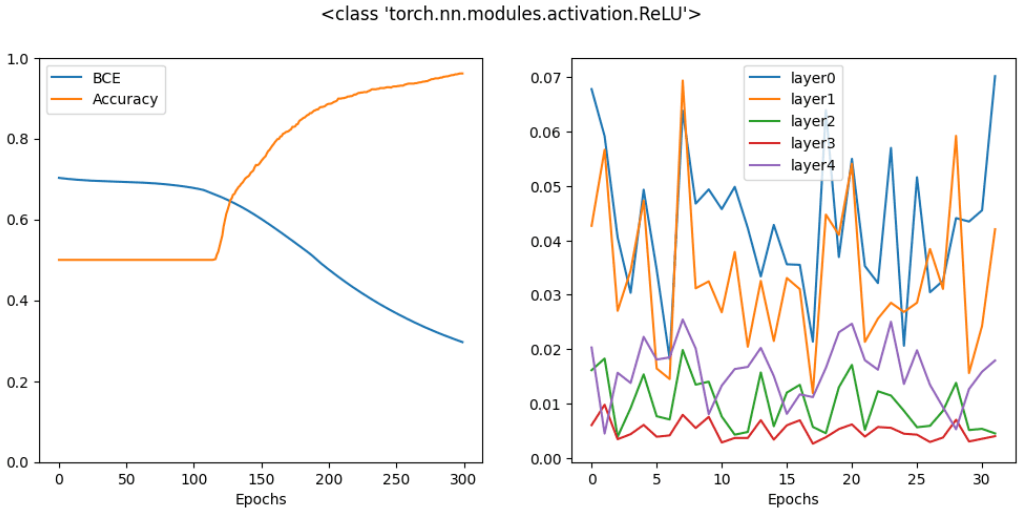


Figure 14.5: The mean absolute gradient of each layer

In the plot above, you can see how the accuracy increases and the cross-entropy loss decreases. At the same time, you can see the gradient of each layer is fluctuating in a similar range, especially you should pay attention to the line corresponding to the first layer and the last layer. This behavior is ideal.

Let's repeat the same with a sigmoid activation:

```
activation = nn.Sigmoid
model = Model(activation=activation)
bce_hist, acc_hist, grad_hist = train_loop(model, X, y)

fig, ax = plt.subplots(1, 2, figsize=(12, 5))
ax[0].plot(bce_hist, label="BCE")
ax[0].plot(acc_hist, label="Accuracy")
ax[0].set_xlabel("Epochs")
ax[0].set_ylim(0, 1)
for n, grads in enumerate(grad_hist):
    ax[1].plot(grads, label="layer"+str(n))
ax[1].set_xlabel("Epochs")
fig.suptitle(str(activation))
```

```
ax[0].legend()
ax[1].legend()
plt.show()
```

Listing 14.7: Repeat the training with a model using sigmoid activation

which the plot is as follows:

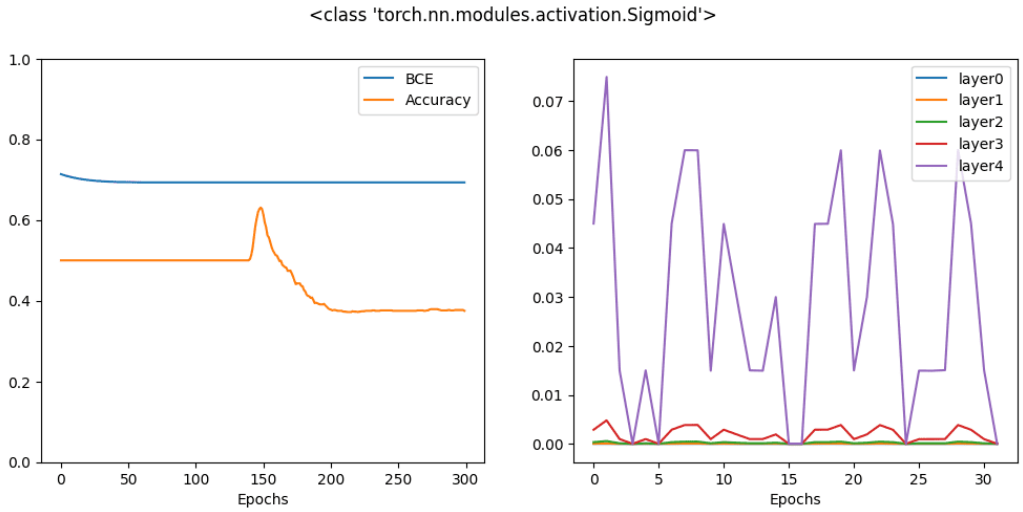


Figure 14.6: Training progress on a model using sigmoid activation

You can see that after 300 epochs, the final result is much worse than ReLU activation. Indeed, you may need much more epochs for this model to converge. The reason can be easily found on the graph at right, which you can see the gradient is significant only for the output layer while all the hidden layers' gradients are virtually zero. This is the *vanishing gradient effect* which is the problem of many neural network models with sigmoid activation function.

The hyperbolic tangent function has a similar shape as sigmoid function but its curve is steeper. Let's see how it behaves:

```
activation = nn.Tanh
model = Model(activation=activation)
bce_hist, acc_hist, grad_hist = train_loop(model, X, y)

fig, ax = plt.subplots(1, 2, figsize=(12, 5))
ax[0].plot(bce_hist, label="BCE")
ax[0].plot(acc_hist, label="Accuracy")
ax[0].set_xlabel("Epochs")
ax[0].set_ylim(0, 1)
for n, grads in enumerate(grad_hist):
    ax[1].plot(grads, label="layer"+str(n))
ax[1].set_xlabel("Epochs")
fig.suptitle(str(activation))
ax[0].legend()
ax[1].legend()
plt.show()
```

Listing 14.8: Repeat the training with a model using hyperbolic tangent activation

Which is:

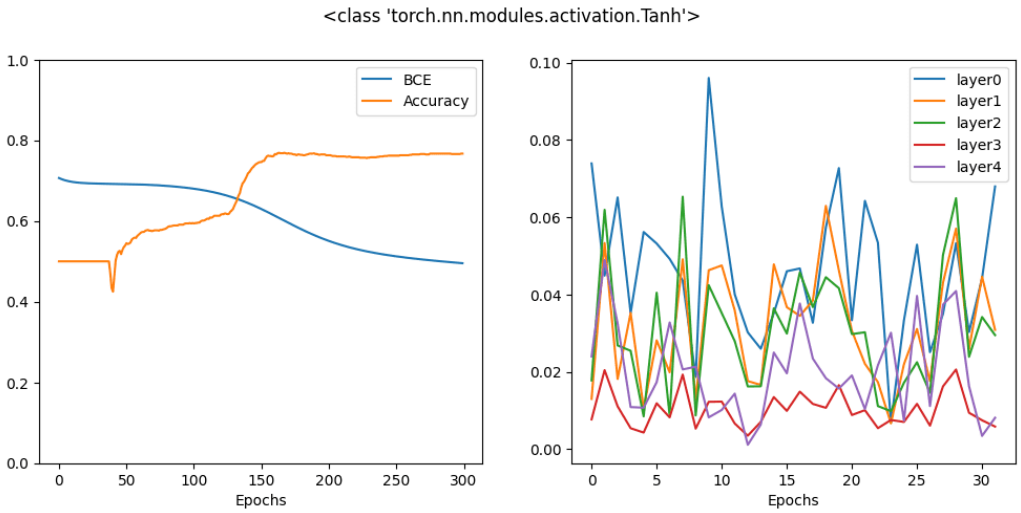


Figure 14.7: Training progress on a model using hyperbolic tangent activation

The result looks better than sigmoid activation but still worse then ReLU. From the gradient plot, you can notice that the gradients at the hidden layers are significant. But in fact, the backward process is often not very effective at propagating the gradient to the input due to the large flat region in tanh function.

This is the reason you see ReLU activation in every neural network model today. Not only because ReLU is simpler and computing the differentiation of it is much faster than the other activation function, but also because it can make the model converge faster.

Indeed, you can do better than ReLU sometimes. In PyTorch, you have a number of ReLU variations. Let's look at two of them. You can compare these three variations of ReLU as follows:

```
import torch
import torch.nn as nn
import matplotlib.pyplot as plt

x = torch.linspace(-8,8,200)
relu = nn.ReLU()(x)
relu6 = nn.ReLU6()(x)
leaky = nn.LeakyReLU()(x)

plt.plot(x, relu, c="purple", lw=2, ls=":", label="ReLU")
plt.plot(x, relu6, c="orange", lw=2, ls="--", alpha=0.5, label="ReLU6")
plt.plot(x, leaky, c="darkblue", lw=2, alpha=0.5, label="LeakyReLU")
plt.legend()
plt.show()
```

Listing 14.9: Plotting three variations of ReLU function

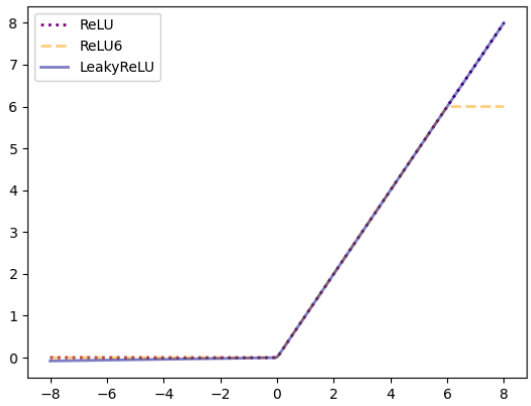


Figure 14.8: The plot of three variations of ReLU function

First is the ReLU6, which is ReLU but cap the function at 6.0 if the input to the function is more than 6.0:

```
activation = nn.ReLU6
model = Model(activation=activation)
bce_hist, acc_hist, grad_hist = train_loop(model, X, y)

fig, ax = plt.subplots(1, 2, figsize=(12, 5))
ax[0].plot(bce_hist, label="BCE")
ax[0].plot(acc_hist, label="Accuracy")
ax[0].set_xlabel("Epochs")
ax[0].set_ylim(0, 1)
for n, grads in enumerate(grad_hist):
    ax[1].plot(grads, label="layer"+str(n))
ax[1].set_xlabel("Epochs")
fig.suptitle(str(activation))
ax[0].legend()
ax[1].legend()
plt.show()
```

Listing 14.10: Repeat the training with a model using ReLU6 activation

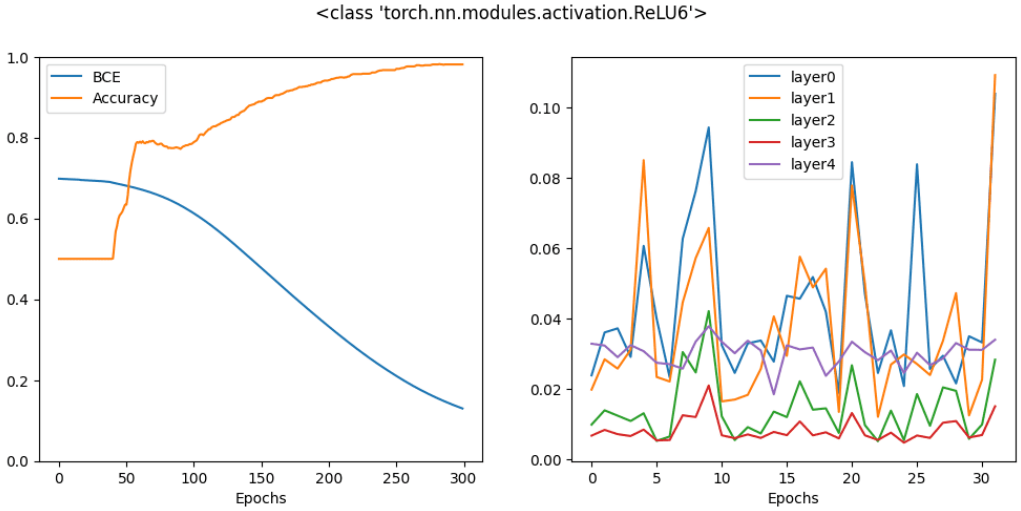


Figure 14.9: Training progress on a model using ReLU6 activation

Next is leaky ReLU, which the negative half of ReLU is no longer flat but a gently slanted line. The rationale behind is to keep a small positive slope at that region.

```
activation = nn.LeakyReLU
model = Model(activation=activation)
bce_hist, acc_hist, grad_hist = train_loop(model, X, y)

fig, ax = plt.subplots(1, 2, figsize=(12, 5))
ax[0].plot(bce_hist, label="BCE")
ax[0].plot(acc_hist, label="Accuracy")
ax[0].set_xlabel("Epochs")
ax[0].set_ylim(0, 1)
for n, grads in enumerate(grad_hist):
    ax[1].plot(grads, label="layer"+str(n))
ax[1].set_xlabel("Epochs")
fig.suptitle(str(activation))
ax[0].legend()
ax[1].legend()
plt.show()
```

Listing 14.11: Repeat the training with a model using leaky ReLU activation

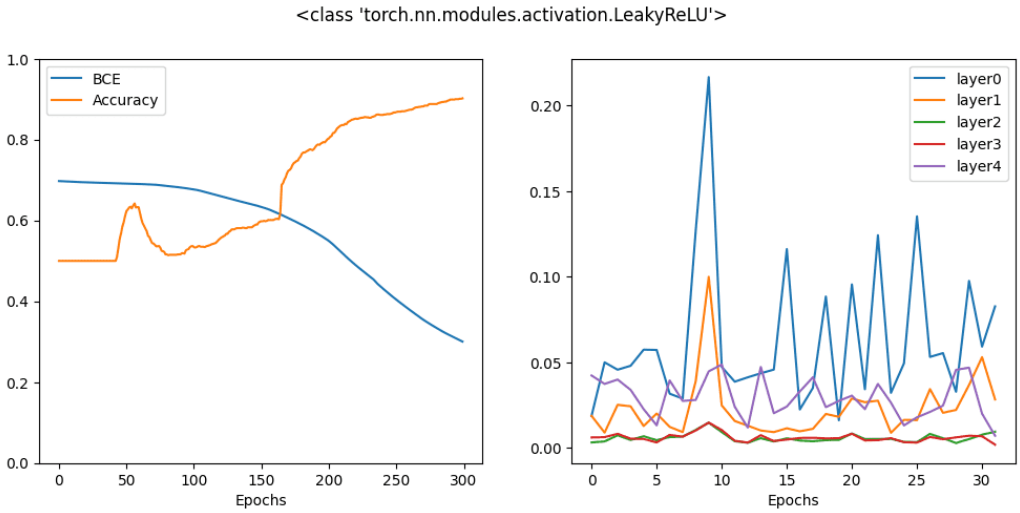


Figure 14.10: Training progress on a model using leaky ReLU activation

You can see that all these variations can give you similar accuracy after 300 epochs but from the history curve, you know some are faster to reach a high accuracy than another. This is because of the interaction between the gradient of an activation function with the optimizer. There's no golden rule that a single activation function works best but the design helps:

- ▷ In backpropagation, passing the loss metric from the output layer all the way to the input layer
- ▷ Maintaining stable gradient calculation under specific condition, e.g., limiting floating point precision. Remember that all floating point calculation involves some round off and the activation function must be robust to such rounding.
- ▷ Providing enough contrast on different inputs such that the backward pass can determine accurate adjustment to the parameter. You can see from Figure 14.3 that sigmoid is just a flattened tanh function (since $\tanh(x) = 2\sigma(2x) - 1$) but you saw how different are the result.

The following is the complete code to generate all the plots above:

```
from sklearn.datasets import make_circles
import matplotlib.pyplot as plt
import torch
import torch.nn as nn
import torch.optim as optim

# Make data: Two circles on x-y plane as a classification problem
X, y = make_circles(n_samples=1000, factor=0.5, noise=0.1)
X = torch.tensor(X, dtype=torch.float32)
y = torch.tensor(y.reshape(-1, 1), dtype=torch.float32)

# Binary classification model
class Model(nn.Module):
    def __init__(self, activation=nn.ReLU):
        super().__init__()
        self.layer0 = nn.Linear(2,5)
```

```
        self.act0 = activation()
        self.layer1 = nn.Linear(5,5)
        self.act1 = activation()
        self.layer2 = nn.Linear(5,5)
        self.act2 = activation()
        self.layer3 = nn.Linear(5,5)
        self.act3 = activation()
        self.layer4 = nn.Linear(5,1)
        self.act4 = nn.Sigmoid()

    def forward(self, x):
        x = self.act0(self.layer0(x))
        x = self.act1(self.layer1(x))
        x = self.act2(self.layer2(x))
        x = self.act3(self.layer3(x))
        x = self.act4(self.layer4(x))
        return x

# train the model and produce history
def train_loop(model, X, y, n_epochs=300, batch_size=32):
    loss_fn = nn.BCELoss()
    optimizer = optim.Adam(model.parameters(), lr=0.0001)
    batch_start = torch.arange(0, len(X), batch_size)

    bce_hist = []
    acc_hist = []
    grad_hist = [[],[],[],[],[]]

    for epoch in range(n_epochs):
        # train model with optimizer
        model.train()
        layer_grad = [[],[],[],[],[]]
        for start in batch_start:
            X_batch = X[start:start+batch_size]
            y_batch = y[start:start+batch_size]
            y_pred = model(X_batch)
            loss = loss_fn(y_pred, y_batch)
            optimizer.zero_grad()
            loss.backward()
            optimizer.step()
            # collect mean absolute value of gradients
            layers = [model.layer0, model.layer1, model.layer2, model.layer3,
                      model.layer4]
            for n,layer in enumerate(layers):
                mean_grad = float(layer.weight.grad.abs().mean())
                layer_grad[n].append(mean_grad)
        # evaluate BCE and accuracy at end of each epoch
        model.eval()
        with torch.no_grad():
            y_pred = model(X)
            bce = float(loss_fn(y_pred, y))
            acc = float((y_pred.round() == y).float().mean())
        bce_hist.append(bce)
        acc_hist.append(acc)
```

```
        for n, grads in enumerate(layer_grad):
            grad_hist[n].append(sum(grads)/len(grads))
        # print metrics every 10 epochs
        if epoch % 10 == 9:
            print("Epoch %d: BCE=%.4f, Accuracy=%.2f%%" % (epoch, bce, acc*100))
    return bce_hist, acc_hist, layer_grad

# pick different activation functions and compare the result visually
for activation in [nn.Sigmoid, nn.Tanh, nn.ReLU, nn.ReLU6, nn.LeakyReLU]:
    model = Model(activation=activation)
    bce_hist, acc_hist, grad_hist = train_loop(model, X, y)

    fig, ax = plt.subplots(1, 2, figsize=(12, 5))
    ax[0].plot(bce_hist, label="BCE")
    ax[0].plot(acc_hist, label="Accuracy")
    ax[0].set_xlabel("Epochs")
    ax[0].set_ylim(0, 1)
    for n, grads in enumerate(grad_hist):
        ax[1].plot(grads, label="layer"+str(n))
    ax[1].set_xlabel("Epochs")
    fig.suptitle(str(activation))
    ax[0].legend()
    ax[1].legend()
    plt.show()
```

Listing 14.12: Experiment on different activation functions

14.4 Further Readings

This section provides more resources on the topic if you are looking to go deeper.

Articles

Vanishing gradient problem. Wikipedia.
https://en.wikipedia.org/wiki/Vanishing_gradient_problem

Sigmoid. PyTorch API.
https://pytorch.org/docs/stable/generated/torch.nn.Sigmoid.html

Tanh. PyTorch API.
https://pytorch.org/docs/stable/generated/torch.nn.Tanh.html

ReLU. PyTorch API.
https://pytorch.org/docs/stable/generated/torch.nn.ReLU.html

ReLU6. PyTorch API.
https://pytorch.org/docs/stable/generated/torch.nn.ReLU6.html

LeakyReLU. PyTorch API.
https://pytorch.org/docs/stable/generated/torch.nn.LeakyReLU.html

14.5 Summary

In this chapter, you discovered how to select activation functions for your PyTorch model. You learned:

- ▷ What are the common activation functions and how they look like
- ▷ How to use activation functions in your PyTorch model
- ▷ What is vanishing gradient problem
- ▷ The impact of activation function to the performance of your model

In the next chapter, you will learn about loss function, another key component in building a deep learning model.

Loss Functions in PyTorch Models

15

The loss metric is very important for neural networks. As all machine learning models are one optimization problem or another, the loss is the objective function to minimize. In neural networks, the optimization is done with gradient descent and backpropagation. But what are loss functions, and how are they affecting your neural networks?

In this chapter, you will learn what loss functions are and delve into some commonly used loss functions and how you can apply them to your neural networks. After reading this chapter, you will learn:

- ▷ What are loss functions, and their role in training neural network models
- ▷ Common loss functions for regression and classification problems
- ▷ How to use loss functions in your PyTorch model

Let's get started!

Overview

This chapter is divided into four sections; they are:

- ▷ What Are Loss Functions?
- ▷ Loss Functions for Regression
- ▷ Loss Functions for Classification
- ▷ Custom Loss Function in PyTorch

15.1 What Are Loss Functions?

In neural networks, loss functions help optimize the performance of the model. They are usually used to measure some penalty that the model incurs on its predictions, such as the deviation of the prediction away from the ground truth label. Loss functions are usually differentiable across their domain (but it is allowed that the gradient is undefined only for very specific points, such as $x = 0$, which is basically ignored in practice). In the training

loop, they are differentiated with respect to parameters, and these gradients are used for your backpropagation and gradient descent steps to optimize your model on the training set.

Loss functions are also slightly different from metrics. While loss functions can tell you the performance of our model, they might not be of direct interest or easily explainable by humans. This is where metrics come in. Metrics such as accuracy are much more useful for humans to understand the performance of a neural network even though they might not be good choices for loss functions since they might not be differentiable.

In the following, let's explore some common loss functions, for regression problems and for classification problems.

15.2 Loss Functions for Regression

In regression problems, the model is to predict a value in a continuous range. Too good to be true that your model can predict the exact value all the time, but it is good enough if the value is close enough. Therefore, you need a loss function to measure how close it is. The farther away from the exact value, the more the loss is your prediction.

One simple function is just to measure the difference between the prediction and the target value. You do not care the value is greater than or less than the target value in finding the difference. Hence, in mathematics, we find $\frac{1}{m}\sum_{i=1}^{m}|\hat{y}_i - y_i|$ with m the number of training examples whereas y_i and $\hat{y}_i$ are the target and predicted values, respectively, averaged over all training examples. This is the mean absolute error (MAE).

The MAE is never negative and would be zero only if the prediction matched the ground truth perfectly. It is an intuitive loss function and might also be used as one of your metrics, specifically for regression problems, since you want to minimize the error in your predictions.

However, absolute value is not differentiable at 0. It is not really a problem because you rarely hit that value. But sometimes people would prefer to use mean square error (MSE) instead. MSE equals to $\frac{1}{m}\sum_{i=1}^{m}(\hat{y}_i - y_i)^2$, which is similar to MAE but use square function in place of absolute value.

It also measures the deviation of the predicted value from the target value. However, the MSE squares this difference (always non-negative since squares of real numbers are always non-negative), which gives it slightly different properties. One property is that the mean squared error tolerates a large number of small errors more than a small number of large errors, which leads to models with fewer outliers or at least outliers that are less severe than models trained with a MAE. This is because the square function amplified large errors, and consequently, the gradient of the resulting MSE is more significant.

Let's look at what the mean absolute error and mean square error loss function looks like graphically:

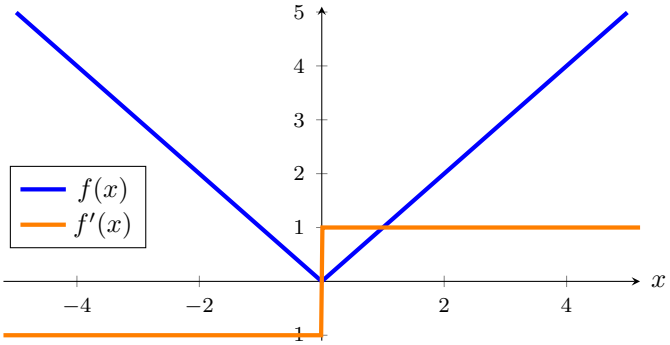


Figure 15.1: Mean absolute error loss function (blue) and gradient (orange)

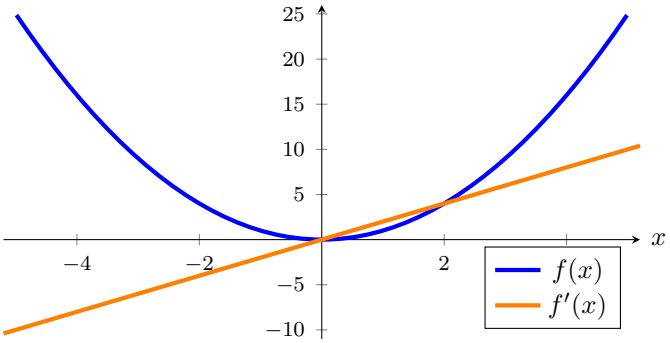


Figure 15.2: Mean square error loss function (blue) and gradient (orange)

Similar to activation functions, you might also be interested in what the gradient of the loss function looks like since you are using the gradient later to do backpropagation to train your model's parameters. You should see that in MSE, larger errors would lead to a larger magnitude for the gradient and a larger loss. Hence, for example, two training examples that deviate from their ground truths by 1 unit would lead to a loss of 2, while a single training example that deviates from its ground truth by 2 units would lead to a loss of 4, hence having a larger impact. This is not the case in MAE.

In PyTorch, you can create MAE and MSE as loss functions using `nn.L1Loss()` and `nn.MSELoss()` respectively. It is named as L1 because the computation of MAE is also called the L1-norm in mathematics. Below is an example of computing the MAE and MSE between two vectors:

```
import torch
import torch.nn as nn

mae = nn.L1Loss()
mse = nn.MSELoss()

predict = torch.tensor([0., 3.])
target = torch.tensor([1., 0.])
```

```
print("MAE: %.3f" % mae(predict, target))
print("MSE: %.3f" % mse(predict, target))
```

Listing 15.1: Computing MSE and MAE

You should get

```
MAE: 2.000
MSE: 5.000
```

Output 15.1: MAE and MSE as computed

MAE is 2.0 because $\frac{1}{2}(|0-1|+|3-0|) = \frac{1}{2}(1+3) = 2$ whereas MSE is 5.0 because $\frac{1}{2}[(0-1)^2+(3-0)^2] = \frac{1}{2}(1+9) = 5$. Notice that in MSE, the second example with a predicted value of 3 and actual value of 0 contributes 90% of the error under the mean squared error vs. 75% under the mean absolute error.

Sometimes, you may see people use root mean squared error (RMSE) as a metric. This will take the square root of MSE. From the perspective of a loss function, MSE and RMSE are equivalent. But from the perspective of the value, the RMSE is in the same unit as the predicted values. If your prediction is money in dollars, both MAE and RMSE give you how much your prediction is away from the true value in dollars in average. But MSE is in unit of squared dollars, which its physical meaning is not intuitive.

15.3 Loss Functions for Classification

For classification problems, there is a small, discrete set of numbers that the output could take. Furthermore, the number used to label-encode the classes is arbitrary and with no semantic meaning (e.g., using the labels 0 for cat, 1 for dog, and 2 for horse does not represent that a dog is half cat and half horse). Therefore, it should not have an impact on the performance of the model.

In a classification problem, the model's output is usually a vector of probability for each category. Often, this vector is usually expected to be *logits*, i.e., real numbers to be transformed to probability using the softmax function, or the output of a softmax activation function.

The cross-entropy between two probability distributions is a measure of the difference between the two probability distributions. Precisely, it is $-\sum_i P(X=x_i)\log Q(X=x_i)$ for probability P and Q. In machine learning, we usually have the probability P provided by the training data and Q predicted by the model, which P is 1 for the correct class and 0 for every other class. The predicted probability Q, however, is usually a floating point valued between 0 and 1. Hence when used for classification problems in machine learning, this formula can be simplified into:

$$\text{categorical cross-entropy} = -\log p_{\text{target}}$$

where p_{target} is the model-predicted probability of the ground-truth class for that particular sample.

Cross-entropy metrics have a negative sign because $\log(x)$ tends to negative infinity as x tends to zero. We want a higher loss when the probability approaches 0 and a lower loss when the probability approaches 1. Graphically,

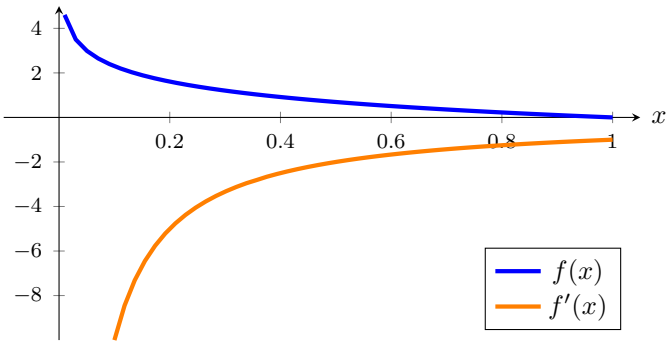


Figure 15.3: Categorical cross-entropy loss function (blue) and gradient (orange)

Notice that the loss is exactly 0 if the probability of the ground truth class is 1 as desired. Also, as the probability of the ground truth class tends to 0, the loss tends to positive infinity as well, hence substantially penalizing bad predictions. You might recognize this loss function for logistic regression, which is similar except the logistic regression loss is specific to the case of binary classes.

Looking at the gradient, you can see that the gradient is generally negative, which is also expected since, to decrease this loss, you would want the probability on the ground truth class to be as high as possible. Recall that gradient descent goes in the opposite direction of the gradient.

In PyTorch, the cross-entropy function is provided by `nn.CrossEntropyLoss()`. It takes the predicted logits and the target as parameter and compute the categorical cross-entropy. Remind that inside the `CrossEntropyLoss()` function, softmax will be applied to the logits hence you should not use softmax activation function at the output layer. Example of using the cross-entropy loss function from PyTorch is as follows:

```
import torch
import torch.nn as nn

ce = nn.CrossEntropyLoss()

logits = torch.tensor([[-1.90, -0.29, -2.30], [-0.29, -1.90, -2.30]])
target = torch.tensor([[0., 1., 0.], [1., 0., 0.]])
print("Cross entropy: %.3f" % ce(logits, target))
```

Listing 15.2: Creating and using cross-entropy loss function

It prints:

```
Cross entropy: 0.288
```

Output 15.2: Cross-entropy as computed

Note the first argument to the cross-entropy loss function is logit, not probabilities. Hence each row does not sum to 1. The second argument, however, is a tensor containing rows of probabilities. If you convert the `logits` tensor above into probability using softmax function, it would be:

```
probs = torch.tensor([[0.15, 0.75, 0.1], [0.75, 0.15, 0.1]])
```

Listing 15.3: Softmax equivalent of the logits

which each row sums to 1.0. This tensor also reveals why the cross-entropy above calculated to be 0.288, which is $-\log(0.75)$.

The other way of calculating the cross-entropy in PyTorch is not to use one-hot encoding in the target but to use the integer indices label:

```
import torch
import torch.nn as nn

ce = nn.CrossEntropyLoss()

logits = torch.tensor([[-1.90, -0.29, -2.30], [-0.29, -1.90, -2.30]])
indices = torch.tensor([1, 0])
print("Cross entropy: %.3f" % ce(logits, indices))
```

Listing 15.4: Cross-entropy loss function on integer labels

This gives you the same cross-entropy of 0.288. Note that,

```
import torch

target = torch.tensor([[0., 1., 0.], [1., 0., 0.]])
indices = torch.argmax(target, dim=1)
print(indices)
```

Listing 15.5: Converting one-hot encoding to integer indices

gives you:

```
tensor([1, 0])
```

Output 15.3: Integer indices as computed using `argmax()` function

This is how PyTorch interprets your target tensor. It is also called *sparse cross-entropy* function in other libraries, to make a distinction that it does not expect a one-hot vector.

Note in PyTorch, you can use `nn.LogSoftmax()` as an activation function. It is to apply softmax on the output of a layer and then take the logarithm on each element. If this is your output layer, you should use `nn.NLLLoss()` (negative log likelihood) as the loss function. Mathematically this duo is same as cross-entropy loss. You can confirm this by checking the code below produced the same output:

```
import torch
import torch.nn as nn

ce = nn.NLLLoss()

# softmax to apply on dimension 1, i.e. per row
logsoftmax = nn.LogSoftmax(dim=1)
```

```
logits = torch.tensor([[-1.90, -0.29, -2.30], [-0.29, -1.90, -2.30]])
pred = logsoftmax(logits)
indices = torch.tensor([1, 0])
print("Cross entropy: %.3f" % ce(pred, indices))
```

Listing 15.6: Using negative log-likelihood function

In case of a classification problem with only two classes, it becomes binary classification. It is special because the model is now a logistic regression model in which there can be only one output instead of a vector of two values. You can still implement binary classification as multiclass classification and use the same cross-entropy function. But if you output x as the probability (between 0 and 1) for the "positive class", it is known that the probability for the "negative class" must be $1 - x$.

In PyTorch, you have `nn.BCELoss()` for binary cross-entropy. It is specialized for binary case. For example:

```
import torch
import torch.nn as nn

bce = nn.BCELoss()

pred = torch.tensor([0.75, 0.25])
target = torch.tensor([1., 0.])
print("Binary cross entropy: %.3f" % bce(pred, target))
```

Listing 15.7: Using binary cross-entropy loss function

This gives you:

```
Binary cross entropy: 0.288
```

Output 15.4: Binary cross-entropy as computed

It is because:

$$\frac{1}{2}[-\log(0.75) + (-\log(1-0.25))] = -\log(0.75) = 0.288$$

Note that in PyTorch, the target label 1 is taken as the "positive class" and label 0 is the "negative class". There should not be other values in the target tensor.

15.4 Custom Loss Function in PyTorch

Notice in above, the loss metric is calculated using an object from `torch.nn` module. The loss metric computed is a PyTorch tensor, so you can differentiate it and start the backpropagation. Therefore, nothing forbid you from creating your own loss function as long as you can compute a tensor based on the model's output.

PyTorch does not give you all the possible loss metrics. For example, mean absolute percentage error is not included. It is like MAE, defined as:

$$\text{MAPE} = \frac{1}{m}\sum_{i=1}^{m}\left|\frac{\hat{y}_i - y_i}{y_i}\right|$$

Sometimes you may prefer to use MAPE. Recall the example on regression on the California housing dataset, the prediction is on the house price. It may make more sense to consider prediction accurate based on percentage difference rather than dollar difference. You can define your MAPE function, just remember to use PyTorch functions to compute, and return a PyTorch tensor.

See the full example below:

```
import copy

import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
from sklearn.model_selection import train_test_split
from sklearn.datasets import fetch_california_housing
from sklearn.preprocessing import StandardScaler

# Read data
data = fetch_california_housing()
X, y = data.data, data.target

# train-test split for model evaluation
X_train_raw, X_test_raw, y_train, y_test = train_test_split(X, y, train_size=0.7,
                                                            shuffle=True)

# Standardizing data
scaler = StandardScaler()
scaler.fit(X_train_raw)
X_train = scaler.transform(X_train_raw)
X_test = scaler.transform(X_test_raw)

# Convert to 2D PyTorch tensors
X_train = torch.tensor(X_train, dtype=torch.float32)
y_train = torch.tensor(y_train, dtype=torch.float32).reshape(-1, 1)
X_test = torch.tensor(X_test, dtype=torch.float32)
y_test = torch.tensor(y_test, dtype=torch.float32).reshape(-1, 1)

# Define the model
model = nn.Sequential(
    nn.Linear(8, 24),
    nn.ReLU(),
    nn.Linear(24, 12),
    nn.ReLU(),
    nn.Linear(12, 6),
    nn.ReLU(),
    nn.Linear(6, 1)
)

# loss function and optimizer
def loss_fn(output, target):
    # MAPE loss
    return torch.mean(torch.abs((target - output) / target))
```

```
optimizer = optim.Adam(model.parameters(), lr=0.0001)

n_epochs = 100   # number of epochs to run
batch_size = 10  # size of each batch
batch_start = torch.arange(0, len(X_train), batch_size)

# Hold the best model
best_mape = np.inf   # init to infinity
best_weights = None

for epoch in range(n_epochs):
    model.train()
    for start in batch_start:
        # take a batch
        X_batch = X_train[start:start+batch_size]
        y_batch = y_train[start:start+batch_size]
        # forward pass
        y_pred = model(X_batch)
        loss = loss_fn(y_pred, y_batch)
        # backward pass
        optimizer.zero_grad()
        loss.backward()
        # update weights
        optimizer.step()
    # evaluate accuracy at end of each epoch
    model.eval()
    y_pred = model(X_test)
    mape = float(loss_fn(y_pred, y_test))
    if mape < best_mape:
        best_mape = mape
        best_weights = copy.deepcopy(model.state_dict())

# restore model and return best accuracy
model.load_state_dict(best_weights)
print("MAPE: %.2f" % best_mape)

model.eval()
with torch.no_grad():
    # Test out inference with 5 samples
    for i in range(5):
        X_sample = X_test_raw[i: i+1]
        X_sample = scaler.transform(X_sample)
        X_sample = torch.tensor(X_sample, dtype=torch.float32)
        y_pred = model(X_sample)
        print(f"{X_test_raw[i]} -> {y_pred[0].numpy()} (expected {y_test[i].numpy()})")
```

Listing 15.8: Training a model using MAPE as custom loss function

Compare to the example in the other chapter, you can see that `loss_fn` now is defined as a custom function. Otherwise, everything is just the same.

15.5 Further Readings

Below are the documentations from PyTorch that give you more details on how the various loss functions are implemented:

APIs

L1Loss. PyTorch API.
https://pytorch.org/docs/stable/generated/torch.nn.L1Loss.html
MSELoss. PyTorch API.
https://pytorch.org/docs/stable/generated/torch.nn.MSELoss.html
CrossEntropyLoss. PyTorch API.
https://pytorch.org/docs/stable/generated/torch.nn.CrossEntropyLoss.html
BCELoss. PyTorch API.
https://pytorch.org/docs/stable/generated/torch.nn.BCELoss.html
NLLLoss. PyTorch API.
https://pytorch.org/docs/stable/generated/torch.nn.NLLLoss.html

15.6 Summary

In this chapter, you have seen loss functions and the role that they play in a neural network. You have also seen some popular loss functions used in regression and classification models, as well as how to implement your own loss function for your PyTorch model. Specifically, you learned:

- ▷ What are loss functions, and why they are important in training
- ▷ Common loss functions for regression and classification problems
- ▷ How to use loss functions in your PyTorch model

In the next chapter, you will learn about the most common regularization technique in deep learning.

Using Dropout Regularization in PyTorch Models

16

Dropout is a simple and powerful regularization technique for neural networks and deep learning models.

In this chapter, you will discover the Dropout regularization technique and how to apply it to your models in PyTorch models.

After reading this chapter, you will know:

- ▷ How the Dropout regularization technique works
- ▷ How to use Dropout on your input layers
- ▷ How to use Dropout on your hidden layers
- ▷ How to tune the dropout level on your problem

Let's get started.

Overview

This chapter is divided into six parts; they are

- ▷ Dropout Regularization for Neural Networks
- ▷ Dropout Regularization in PyTorch
- ▷ Using Dropout on the Input Layer
- ▷ Using Dropout on the Hidden Layers
- ▷ Dropout in Evaluation Mode
- ▷ Tips for Using Dropout

16.1 Dropout Regularization for Neural Networks

Dropout is a regularization technique for neural network models proposed around 2012 to 2014. It is a layer in the neural network. During training of a neural network model, it will take the output from its previous layer, randomly select some neurons and zero them out before passing to the next layer, effectively ignored them. This means that their contribution

to the activation of downstream neurons is temporally removed on the forward pass, and any weight updates are not applied to the neuron on the backward pass. To compensate the dropped neurons, the other neurons will be scaled up in the same percentage as the dropout probability.

When the model is used for inference, dropout layer has no effect, i.e., just pass everything as is to the next layer like an identity function.

Dropout is destructive in training but surprisingly can improve the model's accuracy. As a neural network learns, neuron weights settle into their context within the network. If the model learned a specific pattern and in reality, such pattern is not exactly as seen in training data, the model would be too fragile so failed to predict correctly. Such reliance on a specific context is referred to as complex co-adaptions.

If there are multiple patterns to give the same clue, and the model learned all of them, it should perform better. You can imagine that if neurons are randomly dropped out of the network during training, other neurons will have to step in and handle the representation required to make predictions for the missing neurons. This is believed to result in multiple independent internal representations being learned by the network.

The effect is that the network becomes less sensitive to a specific pattern. This, in turn, results in a network capable of better generalization and less likely to overfit the training data.

16.2 Dropout Regularization in PyTorch

You do not need to randomly select elements from a PyTorch tensor to implement dropout manually. The `nn.Dropout()` layer from PyTorch can be introduced into your model. It is implemented by randomly selecting nodes to be dropped out with a given probability p (e.g., 20%) while in the training loop. In PyTorch, the dropout layer further scale the resulting tensor by a factor of $\frac{1}{1-p}$ so the average tensor value is maintained. Thanks to this scaling, the dropout layer operates at inference will be an identify function (i.e., no effect, simply copy over the input tensor as output tensor). You should make sure to turn the model into inference mode when evaluating the model.

Let's see how to use `nn.Dropout()` in a PyTorch model.

The examples will use the Sonar dataset. This is a binary classification problem that aims to correctly identify rocks and mock-mines from sonar chirp returns. It is a good test dataset for neural networks because all the input values are numerical and have the same scale.

The dataset can be downloaded from the UCI Machine Learning repository[1]. You can place the sonar dataset in your current working directory with the file name *sonar.csv*.

You will evaluate the developed models using scikit-learn with 10-fold cross-validation in order to tease out differences in the results better.

[1] http://archive.ics.uci.edu/ml/machine-learning-databases/undocumented/connectionist-bench/sonar/sonar.all-data

There are 60 input values and a single output value. The input values are standardized before being used in the network. The baseline neural network model has two hidden layers, the first with 60 units and the second with 30. Stochastic gradient descent is used to train the model with a relatively low learning rate and momentum.

The full baseline model is listed below:

```
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
import torch.optim as optim
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import StratifiedKFold

# Read data
data = pd.read_csv("sonar.csv", header=None)
X = data.iloc[:, 0:60]
y = data.iloc[:, 60]

# Label encode the target from string to integer
encoder = LabelEncoder()
encoder.fit(y)
y = encoder.transform(y)

# Convert to 2D PyTorch tensors
X = torch.tensor(X.values, dtype=torch.float32)
y = torch.tensor(y, dtype=torch.float32).reshape(-1, 1)

# Define PyTorch model
class SonarModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.layer1 = nn.Linear(60, 60)
        self.act1 = nn.ReLU()
        self.layer2 = nn.Linear(60, 30)
        self.act2 = nn.ReLU()
        self.output = nn.Linear(30, 1)
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        x = self.act1(self.layer1(x))
        x = self.act2(self.layer2(x))
        x = self.sigmoid(self.output(x))
        return x

# Helper function to train the model and return the validation result
def model_train(model, X_train, y_train, X_val, y_val,
                n_epochs=300, batch_size=16):
    loss_fn = nn.BCELoss()
    optimizer = optim.SGD(model.parameters(), lr=0.01, momentum=0.8)
    batch_start = torch.arange(0, len(X_train), batch_size)
```

```
    model.train()
    for epoch in range(n_epochs):
        for start in batch_start:
            X_batch = X_train[start:start+batch_size]
            y_batch = y_train[start:start+batch_size]
            y_pred = model(X_batch)
            loss = loss_fn(y_pred, y_batch)
            optimizer.zero_grad()
            loss.backward()
            optimizer.step()

    # evaluate accuracy after training
    model.eval()
    y_pred = model(X_val)
    acc = (y_pred.round() == y_val).float().mean()
    acc = float(acc)
    return acc

# run 10-fold cross validation
kfold = StratifiedKFold(n_splits=10, shuffle=True)
accuracies = []
for train, test in kfold.split(X, y):
    # create model, train, and get accuracy
    model = SonarModel()
    acc = model_train(model, X[train], y[train], X[test], y[test])
    print("Accuracy: %.2f" % acc)
    accuracies.append(acc)

# evaluate the model
mean = np.mean(accuracies)
std = np.std(accuracies)
print("Baseline: %.2f%% (+/- %.2f%%)" % (mean*100, std*100))
```

Listing 16.1: Baseline model for binary classification on the sonar data

Running the example generates an estimated classification accuracy of 82%.

```
Accuracy: 0.81
Accuracy: 0.81
Accuracy: 0.76
Accuracy: 0.86
Accuracy: 0.81
Accuracy: 0.90
Accuracy: 0.86
Accuracy: 0.95
Accuracy: 0.65
Accuracy: 0.80
Baseline: 82.12% (+/- 7.78%)
```

Output 16.1: Result of 10-fold cross-validation

16.3 Using Dropout on the Input Layer

Dropout can be applied to input neurons called the visible layer.

In the example below, a new Dropout layer between the input and the first hidden layer was added. The dropout rate is set to 20%, meaning one in five inputs will be randomly excluded from each update cycle.

Continuing from the baseline example above, the code below exercises the same network with input dropout:

```
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
import torch.optim as optim
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import StratifiedKFold

# Read data
data = pd.read_csv("sonar.csv", header=None)
X = data.iloc[:, 0:60]
y = data.iloc[:, 60]

# Label encode the target from string to integer
encoder = LabelEncoder()
encoder.fit(y)
y = encoder.transform(y)

# Convert to 2D PyTorch tensors
X = torch.tensor(X.values, dtype=torch.float32)
y = torch.tensor(y, dtype=torch.float32).reshape(-1, 1)

# Define PyTorch model, with dropout at input
class SonarModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.dropout = nn.Dropout(0.2)
        self.layer1 = nn.Linear(60, 60)
        self.act1 = nn.ReLU()
        self.layer2 = nn.Linear(60, 30)
        self.act2 = nn.ReLU()
        self.output = nn.Linear(30, 1)
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        x = self.dropout(x)
        x = self.act1(self.layer1(x))
        x = self.act2(self.layer2(x))
        x = self.sigmoid(self.output(x))
        return x

# Helper function to train the model and return the validation result
```

```
def model_train(model, X_train, y_train, X_val, y_val,
                n_epochs=300, batch_size=16):
    loss_fn = nn.BCELoss()
    optimizer = optim.SGD(model.parameters(), lr=0.01, momentum=0.8)
    batch_start = torch.arange(0, len(X_train), batch_size)

    model.train()
    for epoch in range(n_epochs):
        for start in batch_start:
            X_batch = X_train[start:start+batch_size]
            y_batch = y_train[start:start+batch_size]
            y_pred = model(X_batch)
            loss = loss_fn(y_pred, y_batch)
            optimizer.zero_grad()
            loss.backward()
            optimizer.step()

    # evaluate accuracy after training
    model.eval()
    y_pred = model(X_val)
    acc = (y_pred.round() == y_val).float().mean()
    acc = float(acc)
    return acc

# run 10-fold cross validation
kfold = StratifiedKFold(n_splits=10, shuffle=True)
accuracies = []
for train, test in kfold.split(X, y):
    # create model, train, and get accuracy
    model = SonarModel()
    acc = model_train(model, X[train], y[train], X[test], y[test])
    print("Accuracy: %.2f" % acc)
    accuracies.append(acc)

# evaluate the model
mean = np.mean(accuracies)
std = np.std(accuracies)
print("Baseline: %.2f%% (+/- %.2f%%)" % (mean*100, std*100))
```

Listing 16.2: Model with dropout layer

Running the example provides a slight drop in classification accuracy, at least on a single test run.

```
Accuracy: 0.62
Accuracy: 0.90
Accuracy: 0.76
Accuracy: 0.62
Accuracy: 0.67
Accuracy: 0.86
Accuracy: 0.90
Accuracy: 0.86
Accuracy: 0.90
```

```
Accuracy: 0.85
Baseline: 79.40% (+/- 11.20%)
```

Output 16.2: Result of 10-fold cross-validation

16.4 Using Dropout on Hidden Layers

Dropout can be applied to hidden neurons in the body of your network model. This is more common.

In the example below, Dropout is applied between the two hidden layers and between the last hidden layer and the output layer. Again a dropout rate of 20% is used:

```
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
import torch.optim as optim
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import StratifiedKFold

# Read data
data = pd.read_csv("sonar.csv", header=None)
X = data.iloc[:, 0:60]
y = data.iloc[:, 60]

# Label encode the target from string to integer
encoder = LabelEncoder()
encoder.fit(y)
y = encoder.transform(y)

# Convert to 2D PyTorch tensors
X = torch.tensor(X.values, dtype=torch.float32)
y = torch.tensor(y, dtype=torch.float32).reshape(-1, 1)

# Define PyTorch model, with dropout at hidden layers
class SonarModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.layer1 = nn.Linear(60, 60)
        self.act1 = nn.ReLU()
        self.dropout1 = nn.Dropout(0.2)
        self.layer2 = nn.Linear(60, 30)
        self.act2 = nn.ReLU()
        self.dropout2 = nn.Dropout(0.2)
        self.output = nn.Linear(30, 1)
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        x = self.act1(self.layer1(x))
        x = self.dropout1(x)
        x = self.act2(self.layer2(x))
```

```
        x = self.dropout2(x)
        x = self.sigmoid(self.output(x))
        return x

# Helper function to train the model and return the validation result
def model_train(model, X_train, y_train, X_val, y_val,
                n_epochs=300, batch_size=16):
    loss_fn = nn.BCELoss()
    optimizer = optim.SGD(model.parameters(), lr=0.01, momentum=0.8)
    batch_start = torch.arange(0, len(X_train), batch_size)

    model.train()
    for epoch in range(n_epochs):
        for start in batch_start:
            X_batch = X_train[start:start+batch_size]
            y_batch = y_train[start:start+batch_size]
            y_pred = model(X_batch)
            loss = loss_fn(y_pred, y_batch)
            optimizer.zero_grad()
            loss.backward()
            optimizer.step()

    # evaluate accuracy after training
    model.eval()
    y_pred = model(X_val)
    acc = (y_pred.round() == y_val).float().mean()
    acc = float(acc)
    return acc

# run 10-fold cross validation
kfold = StratifiedKFold(n_splits=10, shuffle=True)
accuracies = []
for train, test in kfold.split(X, y):
    # create model, train, and get accuracy
    model = SonarModel()
    acc = model_train(model, X[train], y[train], X[test], y[test])
    print("Accuracy: %.2f" % acc)
    accuracies.append(acc)

# evaluate the model
mean = np.mean(accuracies)
std = np.std(accuracies)
print("Baseline: %.2f%% (+/- %.2f%%)" % (mean*100, std*100))
```

Listing 16.3: Modified model with two dropout layers

You can see that in this case, adding dropout layer improved the accuracy a bit.

```
Accuracy: 0.86
Accuracy: 1.00
Accuracy: 0.86
Accuracy: 0.90
Accuracy: 0.90
Accuracy: 0.86
```

```
Accuracy: 0.81
Accuracy: 0.81
Accuracy: 0.70
Accuracy: 0.85
Baseline: 85.50% (+/- 7.36%)
```

Output 16.3: Result of 10-fold cross-validation

16.5 Dropout in Evaluation Mode

Dropout will randomly reset some of the input to zero. If you wonder what happens after you have finished training, the answer is nothing! The PyTorch dropout layer should run like an identity function when the model is in evaluation mode. That's why you have `model.eval()` before you evaluate the model. This is important because the goal of dropout layer is to make sure the network learn enough clues about the input for the prediction, rather than depend on a rare phenomenon in the data. But on inference, you should provide as much information as possible to the model.

16.6 Tips for Using Dropout

The original paper on Dropout provides experimental results on a suite of standard machine learning problems. As a result, they provide a number of useful heuristics to consider when using Dropout in practice.

- ▷ Generally, use a small dropout value of 20%–50% of neurons, with 20% providing a good starting point. A probability too low has minimal effect, and a value too high results in under-learning by the network.
- ▷ Use a larger network. You are likely to get better performance when Dropout is used on a larger network, giving the model more of an opportunity to learn independent representations.
- ▷ Use Dropout on incoming (visible) as well as hidden units. Application of Dropout at each layer of the network has shown good results.
- ▷ Use a large learning rate with decay or a large momentum. Increase your learning rate by a factor of 10 to 100 or use a high momentum value of 0.9 or 0.99. This is contrary to the intuition, but the authors argue that dropout introduced a lot of noise and this can help speed up learning.
- ▷ Constrain the size of network weights. A large learning rate can result in very large network weights. Imposing a constraint on the size of network weights, such as max-norm regularization, with a size of 4 or 5 has been shown to improve results.

16.7 Further Readings

Below are resources you can use to learn more about Dropout in neural networks and deep learning models.

Papers

Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. "Dropout: A Simple Way to Prevent Neural Networks from Overfitting". *Journal of Machine Learning Research*, 15(56), 2014, pp. 1929–1958.
https://jmlr.org/papers/v15/srivastava14a.html

Geoffrey E. Hinton, Nitish Srivastava, Alex Krizhevsky, Ilya Sutskever, and Ruslan R. Salakhutdinov. "Improving neural networks by preventing co-adaptation of feature detectors", 2012.
https://arxiv.org/abs/1207.0580

Online materials

Connectionist Bench (Snoar, Mines vs. Rocks) Data Set. UCI Machine Learning Repository.
https://archive.ics.uci.edu/ml/datasets/Connectionist+Bench+(Sonar,+Mines+vs.+Rocks)

How does the dropout method work in deep learning? Quora.
https://www.quora.com/How-does-the-dropout-method-work-in-deep-learning

Dropout. PyTorch API.
https://pytorch.org/docs/stable/generated/torch.nn.Dropout.html

16.8 Summary

In this chapter, you discovered the Dropout regularization technique for deep learning models. You learned:

- ▷ What Dropout is and how it works
- ▷ How you can use Dropout on your own deep learning models.
- ▷ Tips for getting the best results from Dropout on your own models.

In the next chapter, you will learn about how learning rate schedule can speed up your training.

Using Learning Rate Schedule in PyTorch Training

17

Training a neural network or large deep learning model is a difficult optimization task.

The classical algorithm to train neural networks is called stochastic gradient descent. It has been well established that you can achieve increased performance and faster training on some problems by using a learning rate that changes during training.

In this chapter, you will discover what is learning rate schedule and how you can use different learning rate schedules for your neural network models in PyTorch.

After reading this chapter, you will know:

- ▷ The role of learning rate schedule in model training
- ▷ How to use learning rate schedule in PyTorch training loop
- ▷ How to setup your own learning rate schedule

Let's get started.

Overview

This chapter is divided into three parts; they are

- ▷ Learning Rate Schedule for Training Models
- ▷ Applying Learning Rate Schedule in PyTorch Training
- ▷ Custom Learning Rate Schedules

17.1 Learning Rate Schedule for Training Models

Gradient descent is an algorithm of numerical optimization. What it does is to update parameters using the formula:

$$w := w - \alpha \frac{dy}{dw}$$

In this formula, w is the parameter, e.g., the weight in a neural network, and y is the objective, e.g., the loss function. What it does is to move w to the direction that you can minimize y.

The direction is provided by the differentiation, $\frac{dy}{dw}$, but how much you should move w is controlled by the *learning rate* α.

An easy start is to use a constant learning rate in gradient descent algorithm. But you can do better with a *learning rate schedule*. A schedule is to make learning rate adaptive to the gradient descent optimization procedure, so you can increase performance and reduce training time.

In the neural network training process, data is feed into the network in batches, with many batches in one epoch. Each batch triggers one training step, which the gradient descent algorithm updates the parameters once. However, usually the learning rate schedule is updated once for each training epoch only.

You can update the learning rate as frequent as each step but usually it is updated once per epoch because you want to know how the network performs in order to determine how the learning rate should update. Regularly, a model is evaluated with validation dataset once per epoch.

There are multiple ways of making learning rate adaptive. At the beginning of training, you may prefer a larger learning rate so you improve the network coarsely to speed up the progress. In a very complex neural network model, you may also prefer to gradually increase the learning rate at the beginning because you need the network to explore on the different dimensions of prediction. At the end of training, however, you always want to have the learning rate smaller. Since at that time, you are about to get the best performance from the model and it is easy to overshoot if the learning rate is large.

Therefore, the simplest and perhaps most used adaptation of the learning rate during training are techniques that reduce the learning rate over time. These have the benefit of making large changes at the beginning of the training procedure when larger learning rate values are used and decreasing the learning rate so that a smaller rate and, therefore, smaller training updates are made to weights later in the training procedure.

This has the effect of quickly learning good weights early and fine-tuning them later.

Next, let's look at how you can setup learning rate schedules in PyTorch.

17.2 Applying Learning Rate Schedules in PyTorch Training

In PyTorch, a model is updated by an optimizer and learning rate is a parameter of the optimizer. Learning rate schedule is an algorithm to update the learning rate in an optimizer.

Below is an example of creating a learning rate schedule:

```
import torch
import torch.optim as optim
import torch.optim.lr_scheduler as lr_scheduler

scheduler = lr_scheduler.LinearLR(optimizer,
                                  start_factor=1.0, end_factor=0.3, total_iters=10)
```

Listing 17.1: Setting up a learning rate schedule

There are many learning rate scheduler provided by PyTorch in `torch.optim.lr_scheduler` submodule. All the scheduler needs the optimizer to update as first argument. Depends on the scheduler, you may need to provide more arguments to setup one.

Let's start with an example model. In below, a model is to solve the ionosphere binary classification problem. This is a small dataset that you can download from the UCI Machine Learning repository[1]. Place the data file in your working directory with the filename `ionosphere.csv`.

The ionosphere dataset is good for practicing with neural networks because all the input values are small numerical values of the same scale.

A small neural network model is constructed with a single hidden layer with 34 neurons, using the ReLU activation function. The output layer has a single neuron and uses the sigmoid activation function in order to output probability-like values.

Plain stochastic gradient descent algorithm is used, with a fixed learning rate 0.1. The model is trained for 50 epochs. The state parameters of an optimizer can be found in `optimizer.param_groups`; which the learning rate is a floating point value at `optimizer.param_groups[0]["lr"]`. At the end of each epoch, the learning rate from the optimizer is printed.

The complete example is listed below.

```
import pandas as pd
import torch
import torch.nn as nn
import torch.optim as optim
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split

# load dataset, split into input (X) and output (y) variables
dataframe = pd.read_csv("ionosphere.csv", header=None)
dataset = dataframe.values
X = dataset[:,0:34].astype(float)
y = dataset[:,34]

# encode class values as integers
encoder = LabelEncoder()
encoder.fit(y)
y = encoder.transform(y)

# convert into PyTorch tensors
X = torch.tensor(X, dtype=torch.float32)
y = torch.tensor(y, dtype=torch.float32).reshape(-1, 1)

# train-test split for evaluation of the model
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size=0.7, shuffle=True)

# create model
model = nn.Sequential(
```

[1] http://archive.ics.uci.edu/ml/machine-learning-databases/ionosphere/ionosphere.data

```
    nn.Linear(34, 34),
    nn.ReLU(),
    nn.Linear(34, 1),
    nn.Sigmoid()
)

# Train the model
n_epochs = 50
batch_size = 24
batch_start = torch.arange(0, len(X_train), batch_size)
lr = 0.1
loss_fn = nn.BCELoss()
optimizer = optim.SGD(model.parameters(), lr=lr)
model.train()
for epoch in range(n_epochs):
    for start in batch_start:
        X_batch = X_train[start:start+batch_size]
        y_batch = y_train[start:start+batch_size]
        y_pred = model(X_batch)
        loss = loss_fn(y_pred, y_batch)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
    print("Epoch %d: SGD lr=%.4f" % (epoch, optimizer.param_groups[0]["lr"]))

# evaluate accuracy after training
model.eval()
y_pred = model(X_test)
acc = (y_pred.round() == y_test).float().mean()
acc = float(acc)
print("Model accuracy: %.2f%%" % (acc*100))
```

Listing 17.2: Model training with fixed learning rate

Running this model produces:

```
Epoch 0: SGD lr=0.1000
Epoch 1: SGD lr=0.1000
Epoch 2: SGD lr=0.1000
Epoch 3: SGD lr=0.1000
Epoch 4: SGD lr=0.1000
...
Epoch 45: SGD lr=0.1000
Epoch 46: SGD lr=0.1000
Epoch 47: SGD lr=0.1000
Epoch 48: SGD lr=0.1000
Epoch 49: SGD lr=0.1000
Model accuracy: 86.79%
```

Output 17.1: Training progress with learning rate reported

You can confirm that the learning rate didn't change over the entire training process. Let's make the training process start with a larger learning rate and end with a smaller rate. To

introduce a learning rate scheduler, you need to run its `step()` function in the training loop. The code above is modified into the following:

```
import pandas as pd
import torch
import torch.nn as nn
import torch.optim as optim
import torch.optim.lr_scheduler as lr_scheduler
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split

# load dataset, split into input (X) and output (y) variables
dataframe = pd.read_csv("ionosphere.csv", header=None)
dataset = dataframe.values
X = dataset[:,0:34].astype(float)
y = dataset[:,34]

# encode class values as integers
encoder = LabelEncoder()
encoder.fit(y)
y = encoder.transform(y)

# convert into PyTorch tensors
X = torch.tensor(X, dtype=torch.float32)
y = torch.tensor(y, dtype=torch.float32).reshape(-1, 1)

# train-test split for evaluation of the model
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size=0.7, shuffle=True)

# create model
model = nn.Sequential(
    nn.Linear(34, 34),
    nn.ReLU(),
    nn.Linear(34, 1),
    nn.Sigmoid()
)

# Train the model
n_epochs = 50
batch_size = 24
batch_start = torch.arange(0, len(X_train), batch_size)
lr = 0.1
loss_fn = nn.BCELoss()
optimizer = optim.SGD(model.parameters(), lr=lr)
scheduler = lr_scheduler.LinearLR(optimizer,
                                  start_factor=1.0, end_factor=0.5, total_iters=30)
model.train()
for epoch in range(n_epochs):
    for start in batch_start:
        X_batch = X_train[start:start+batch_size]
        y_batch = y_train[start:start+batch_size]
        y_pred = model(X_batch)
        loss = loss_fn(y_pred, y_batch)
```

```
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
    before_lr = optimizer.param_groups[0]["lr"]
    scheduler.step()
    after_lr = optimizer.param_groups[0]["lr"]
    print("Epoch %d: SGD lr %.4f -> %.4f" % (epoch, before_lr, after_lr))

# evaluate accuracy after training
model.eval()
y_pred = model(X_test)
acc = (y_pred.round() == y_test).float().mean()
acc = float(acc)
print("Model accuracy: %.2f%%" % (acc*100))
```

Listing 17.3: Model training with learning rate scheduler

It prints:

```
Epoch 0: SGD lr 0.1000 -> 0.0983
Epoch 1: SGD lr 0.0983 -> 0.0967
Epoch 2: SGD lr 0.0967 -> 0.0950
Epoch 3: SGD lr 0.0950 -> 0.0933
Epoch 4: SGD lr 0.0933 -> 0.0917
...
Epoch 28: SGD lr 0.0533 -> 0.0517
Epoch 29: SGD lr 0.0517 -> 0.0500
Epoch 30: SGD lr 0.0500 -> 0.0500
Epoch 31: SGD lr 0.0500 -> 0.0500
...
Epoch 48: SGD lr 0.0500 -> 0.0500
Epoch 49: SGD lr 0.0500 -> 0.0500
Model accuracy: 88.68%
```

Output 17.2: Change in learning rates

First thing to notice is the improved accuracy, which is the reason why you would be interested in a different learning rate schedule.

In the above, `LinearLR()` is used. It is a linear rate scheduler and it takes three additional parameters, the `start_factor`, `end_factor`, and `total_iters`. You set `start_factor` to 1.0, `end_factor` to 0.5, and `total_iters` to 30, therefore it will make a multiplicative factor decrease from 1.0 to 0.5, in 30 equal steps. After 30 steps, the factor will stay at 0.5. This factor is then multiplied to the original learning rate at the optimizer. Hence you will see the learning rate decreased from $0.1 \times 1.0 = 0.1$ to $0.1 \times 0.5 = 0.05$.

Besides `LinearLR()`, you can also use `ExponentialLR()`, its syntax is:

```
scheduler = lr_scheduler.ExponentialLR(optimizer, gamma=0.99)
```

Listing 17.4: Using exponential learning rate scheduler

If you replaced `LinearLR()` with this, you will see the learning rate updated as follows:

```
Epoch 0: SGD lr 0.1000 -> 0.0990
Epoch 1: SGD lr 0.0990 -> 0.0980
Epoch 2: SGD lr 0.0980 -> 0.0970
Epoch 3: SGD lr 0.0970 -> 0.0961
Epoch 4: SGD lr 0.0961 -> 0.0951
...
Epoch 45: SGD lr 0.0636 -> 0.0630
Epoch 46: SGD lr 0.0630 -> 0.0624
Epoch 47: SGD lr 0.0624 -> 0.0617
Epoch 48: SGD lr 0.0617 -> 0.0611
Epoch 49: SGD lr 0.0611 -> 0.0605
```

Output 17.3: Learning rate updated in training loop

In which the learning rate is updated by multiplying with a constant factor `gamma` in each scheduler update.

17.3 Custom Learning Rate Schedules

There is no general rule that a particular learning rate schedule works the best. Sometimes, you like to have a special learning rate schedule that PyTorch didn't provide. A custom learning rate schedule can be defined using a custom function. For example, you want to have a learning rate that:

$$lr_n = \frac{lr_0}{1 + \alpha n}$$

on epoch n, which lr_0 is the initial learning rate, at epoch 0, and α is a constant. You can implement a function that given the epoch n calculate learning rate lr_n:

```
def lr_lambda(epoch):
    # LR to be 0.1 * (1/1+0.01*epoch)
    base_lr = 0.1
    factor = 0.01
    return base_lr/(1+factor*epoch)
```

Listing 17.5: Function to compute custom learning rate

Then, you can setup a `LambdaLR()` to update the learning rate according to this function:

```
scheduler = lr_scheduler.LambdaLR(optimizer, lr_lambda)
```

Listing 17.6: Setting up learning rate scheduler using a custom function

Modifying the previous example to use `LambdaLR()`, you have the following:

```
import pandas as pd
import torch
import torch.nn as nn
import torch.optim as optim
import torch.optim.lr_scheduler as lr_scheduler
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
```

```
# load dataset, split into input (X) and output (y) variables
dataframe = pd.read_csv("ionosphere.csv", header=None)
dataset = dataframe.values
X = dataset[:,0:34].astype(float)
y = dataset[:,34]

# encode class values as integers
encoder = LabelEncoder()
encoder.fit(y)
y = encoder.transform(y)

# convert into PyTorch tensors
X = torch.tensor(X, dtype=torch.float32)
y = torch.tensor(y, dtype=torch.float32).reshape(-1, 1)

# train-test split for evaluation of the model
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size=0.7, shuffle=True)

# create model
model = nn.Sequential(
    nn.Linear(34, 34),
    nn.ReLU(),
    nn.Linear(34, 1),
    nn.Sigmoid()
)

def lr_lambda(epoch):
    # LR to be 0.1 * (1/1+0.01*epoch)
    base_lr = 0.1
    factor = 0.01
    return base_lr/(1+factor*epoch)

# Train the model
n_epochs = 50
batch_size = 24
batch_start = torch.arange(0, len(X_train), batch_size)
lr = 0.1
loss_fn = nn.BCELoss()
optimizer = optim.SGD(model.parameters(), lr=lr)
scheduler = lr_scheduler.LambdaLR(optimizer, lr_lambda)
model.train()
for epoch in range(n_epochs):
    for start in batch_start:
        X_batch = X_train[start:start+batch_size]
        y_batch = y_train[start:start+batch_size]
        y_pred = model(X_batch)
        loss = loss_fn(y_pred, y_batch)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
    before_lr = optimizer.param_groups[0]["lr"]
    scheduler.step()
    after_lr = optimizer.param_groups[0]["lr"]
```

```
    print("Epoch %d: SGD lr %.4f -> %.4f" % (epoch, before_lr, after_lr))

# evaluate accuracy after training
model.eval()
y_pred = model(X_test)
acc = (y_pred.round() == y_test).float().mean()
acc = float(acc)
print("Model accuracy: %.2f%%" % (acc*100))
```

Listing 17.7: Model training with custom learning rate schedule

Which produces:

```
Epoch 0: SGD lr 0.0100 -> 0.0099
Epoch 1: SGD lr 0.0099 -> 0.0098
Epoch 2: SGD lr 0.0098 -> 0.0097
Epoch 3: SGD lr 0.0097 -> 0.0096
Epoch 4: SGD lr 0.0096 -> 0.0095
...
Epoch 45: SGD lr 0.0069 -> 0.0068
Epoch 46: SGD lr 0.0068 -> 0.0068
Epoch 47: SGD lr 0.0068 -> 0.0068
Epoch 48: SGD lr 0.0068 -> 0.0067
Epoch 49: SGD lr 0.0067 -> 0.0067
```

Output 17.4: Learning rate update in training loop

Note that although the function provided to `LambdaLR()` assumes an argument `epoch`, it is not tied to the epoch in the training loop but simply counts how many times you invoked `scheduler.step()`.

17.4 Tips for Using Learning Rate Schedules

This section lists some tips and tricks to consider when using learning rate schedules with neural networks.

- ▷ *Increase the initial learning rate.* Because the learning rate will very likely decrease, start with a larger value to decrease from. A larger learning rate will result in a lot larger changes to the weights, at least in the beginning, allowing you to benefit from the fine-tuning later.
- ▷ *Use a large momentum.* Many optimizers can consider momentum. Using a larger momentum value will help the optimization algorithm continue to make updates in the right direction when your learning rate shrinks to small values.
- ▷ *Experiment with different schedules.* It will not be clear which learning rate schedule to use, so try a few with different configuration options and see what works best on your problem. Also, try schedules that change exponentially and even schedules that respond to the accuracy of your model on the training or test datasets.

17.5 Further Readings

Below is the documentation for more details on using learning rates in PyTorch:

Articles

Ionosphere Data Set. UCI Machine Learning Repository.
https://archive.ics.uci.edu/ml/datasets/Ionosphere
How to adjust learning rate. PyTorch documentation.
https://pytorch.org/docs/stable/optim.html#how-to-adjust-learning-rate

17.6 Summary

In this chapter, you discovered learning rate schedules for training neural network models. After reading this chapter, you learned:

- ▷ How learning rate affects your model training
- ▷ How to setup learning rate schedule in PyTorch
- ▷ How to create a custom learning rate schedule

In the next chapter, you will learn about the DataLoader class that comes with PyTorch and how it can help you make a more concise training loop.

Training a PyTorch Model with DataLoader and Dataset

18

When you build and train a PyTorch deep learning model, you can provide the training data in several different ways. Ultimately, a PyTorch model works like a function that takes a PyTorch tensor and returns you another tensor. You have a lot of freedom in how to get the input tensors. Probably the easiest is to prepare a large tensor of the entire dataset and extract a small batch from it in each training step. But you will see that using the `DataLoader` can save you a few lines of code in dealing with data.

In this chapter, you will see how you can use the Data and DataLoader in PyTorch. After finishing this chapter, you will learn:

- ▷ How to create and use DataLoader to train your PyTorch model
- ▷ How to use Data class to generate data on the fly

Let's get started.

Overview

This chapter is divided into three parts; they are:

- ▷ What is `DataLoader`?
- ▷ Using `DataLoader` in a Training Loop

18.1 What is DataLoader?

To train a deep learning model, you need data. Usually data is available as a dataset. In a dataset, there are a lot of data sample or instances. You can ask the model to take one sample at a time but usually you would let the model to process one batch of several samples. You may create a batch by extracting a slice from the dataset, using the slicing syntax on the tensor. For a better quality of training, you may also want to shuffle the entire dataset on each epoch so no two batch would be the same in the entire training loop. Sometimes, you may introduce *data augmentation* to manually introduce more variance to the data. This is common for image-related tasks, which you can randomly tilt or zoom the image a bit to generate a lot of data sample from a few images.

You can imagine there can be a lot of code to write to do all these. But it is much easier with the `DataLoader`.

The following is an example of how to create a `DataLoader` and take a batch from it. In this example, the sonar dataset[1] is used and ultimately, it is converted into PyTorch tensors and passed on to `DataLoader`:

```
import pandas as pd
import torch
from torch.utils.data import DataLoader
from sklearn.preprocessing import LabelEncoder

# Read data, convert to NumPy arrays
data = pd.read_csv("sonar.csv", header=None)
X = data.iloc[:, 0:60].values
y = data.iloc[:, 60].values

# encode class values as integers
encoder = LabelEncoder()
encoder.fit(y)
y = encoder.transform(y)

# convert into PyTorch tensors
X = torch.tensor(X, dtype=torch.float32)
y = torch.tensor(y, dtype=torch.float32).reshape(-1, 1)

# create DataLoader, then take one batch
loader = DataLoader(list(zip(X,y)), shuffle=True, batch_size=16)
for X_batch, y_batch in loader:
    print(X_batch, y_batch)
    break
```

Listing 18.1: Loading the sonar dataset and convert to PyTorch tensors

You can see from the output of above that `X_batch` and `y_batch` are PyTorch tensors. The `loader` is an instance of `DataLoader` class which can work like an iterable. Each time you read from it, you get a batch of features and targets from the original dataset.

When you create a `DataLoader` instance, you need to provide a list of sample pairs. Each sample pair is one data sample of feature and the corresponding target. A list is required because `DataLoader` expect to use `len()` to find the total size of the dataset and using array index to retrieve a particular sample. The batch size is a parameter to `DataLoader` so it knows how to create a batch from the entire dataset. You should almost always use `shuffle=True` so every time you load the data, the samples are shuffled. It is useful for training because in each epoch, you are going to read every batch once. When you proceed from one epoch to another, as `DataLoader` knows you depleted all the batches, it will re-shuffle so you get a new combination of samples.

[1] http://archive.ics.uci.edu/ml/datasets/connectionist+bench+(sonar,+mines+vs.+rocks)

18.2 Using DataLoader in a Training Loop

The following is an example to make use of `DataLoader` in a training loop:

```
import torch
import torch.nn as nn
import torch.optim as optim
from sklearn.model_selection import train_test_split

# train-test split for evaluation of the model
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size=0.7, shuffle=True)

# setup DataLoader for training set
loader = DataLoader(list(zip(X_train, y_train)), shuffle=True, batch_size=16)

# create model
model = nn.Sequential(
    nn.Linear(60, 60),
    nn.ReLU(),
    nn.Linear(60, 30),
    nn.ReLU(),
    nn.Linear(30, 1),
    nn.Sigmoid()
)

# Train the model
n_epochs = 200
loss_fn = nn.BCELoss()
optimizer = optim.SGD(model.parameters(), lr=0.1)
model.train()
for epoch in range(n_epochs):
    for X_batch, y_batch in loader:
        y_pred = model(X_batch)
        loss = loss_fn(y_pred, y_batch)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

# evaluate accuracy after training
model.eval()
y_pred = model(X_test)
acc = (y_pred.round() == y_test).float().mean()
acc = float(acc)
print("Model accuracy: %.2f%%" % (acc*100))
```

Listing 18.2: Training a model with `DataLoader`

You can see that once you created the `DataLoader` instance, the training loop can only be easier. In the above, only the training set is packaged with a `DataLoader` because you need to loop through it in batches. You can also create a `DataLoader` for the test set and use it for model evaluation, but since the accuracy is computed over the entire test set rather than in a batch, the benefit of `DataLoader` is not significant.

Putting everything together, below is the complete code.

```
import pandas as pd
import torch
import torch.nn as nn
import torch.optim as optim
from sklearn.model_selection import train_test_split
from torch.utils.data import DataLoader
from sklearn.preprocessing import LabelEncoder

# Read data, convert to NumPy arrays
data = pd.read_csv("sonar.csv", header=None)
X = data.iloc[:, 0:60].values
y = data.iloc[:, 60].values

# encode class values as integers
encoder = LabelEncoder()
encoder.fit(y)
y = encoder.transform(y)

# convert into PyTorch tensors
X = torch.tensor(X, dtype=torch.float32)
y = torch.tensor(y, dtype=torch.float32).reshape(-1, 1)

# train-test split for evaluation of the model
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size=0.7, shuffle=True)

# set up DataLoader for training set
loader = DataLoader(list(zip(X_train, y_train)), shuffle=True, batch_size=16)

# create model
model = nn.Sequential(
    nn.Linear(60, 60),
    nn.ReLU(),
    nn.Linear(60, 30),
    nn.ReLU(),
    nn.Linear(30, 1),
    nn.Sigmoid()
)

# Train the model
n_epochs = 200
loss_fn = nn.BCELoss()
optimizer = optim.SGD(model.parameters(), lr=0.1)
model.train()
for epoch in range(n_epochs):
    for X_batch, y_batch in loader:
        y_pred = model(X_batch)
        loss = loss_fn(y_pred, y_batch)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

# evaluate accuracy after training
model.eval()
y_pred = model(X_test)
```

```
acc = (y_pred.round() == y_test).float().mean()
acc = float(acc)
print("Model accuracy: %.2f%%" % (acc*100))
```

Listing 18.3: Reading data from CSV and train a model with DataLoader

18.3 Create Data Iterator using Dataset Class

In PyTorch, there is a `Dataset` class that can be tightly coupled with the `DataLoader` class. Recall that `DataLoader` expects its first argument can work with `len()` and with array index. The `Dataset` class is a base class for this. The reason you may want to use `Dataset` class is there are some special handling before you can get the data sample. For example, data should be read from database or disk and you only want to keep a few samples in memory rather than prefetch everything. Another example is to perform real-time preprocessing of data, such as random augmentation that is common in image tasks.

To use `Dataset` class, you just subclass from it and implement two member functions. Below is an example:

```
from torch.utils.data import Dataset

class SonarDataset(Dataset):
    def __init__(self, X, y):
        # convert into PyTorch tensors and remember them
        self.X = torch.tensor(X, dtype=torch.float32)
        self.y = torch.tensor(y, dtype=torch.float32)

    def __len__(self):
        # this should return the size of the dataset
        return len(self.X)

    def __getitem__(self, idx):
        # this should return one sample from the dataset
        features = self.X[idx]
        target = self.y[idx]
        return features, target
```

Listing 18.4: Creating a `Dataset`

This is not the most powerful way to use `Dataset` but simple enough to demonstrate how it works. With this, you can create a `DataLoader` and use it for model training. Modifying from the previous example, you have the following:

```
...

# setup DataLoader for training set
dataset = SonarDataset(X_train, y_train)
loader = DataLoader(dataset, shuffle=True, batch_size=16)

# create model
model = nn.Sequential(
```

```
    nn.Linear(60, 60),
    nn.ReLU(),
    nn.Linear(60, 30),
    nn.ReLU(),
    nn.Linear(30, 1),
    nn.Sigmoid()
)

# Train the model
n_epochs = 200
loss_fn = nn.BCELoss()
optimizer = optim.SGD(model.parameters(), lr=0.1)
model.train()
for epoch in range(n_epochs):
    for X_batch, y_batch in loader:
        y_pred = model(X_batch)
        loss = loss_fn(y_pred, y_batch)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

# evaluate accuracy after training
model.eval()
y_pred = model(torch.tensor(X_test, dtype=torch.float32))
y_test = torch.tensor(y_test, dtype=torch.float32)
acc = (y_pred.round() == y_test).float().mean()
acc = float(acc)
print("Model accuracy: %.2f%%" % (acc*100))
```

Listing 18.5: Using `DataLoader` with `Dataset`

You setup `dataset` as an instance of `SonarDataset` which you implemented the `__len__()` and `__getitem__()` functions. This is used in place of the list in the previous example to setup the `DataLoader` instance. Afterward, everything is the same in the training loop. Note that you still use PyTorch tensors directly for the test set in the example.

In the `__getitem__()` function, you take an integer that works like an array index and returns a pair, the features and the target. You can implement anything in this function: Run some code to generate a synthetic data sample, read data on the fly from the internet, or add random variations to the data. You will also find it useful in the situation that you cannot keep the entire dataset in memory, so you can load only the data samples that you need it.

In fact, since you created a PyTorch dataset, you don't need to use scikit-learn to split data into training set and test set. In `torch.utils.data` submodule, you have a function `random_split()` that works with `Dataset` class for the same purpose. A full example is below:

```
import pandas as pd
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader, random_split, default_collate
from sklearn.preprocessing import LabelEncoder
```

```
# Read data, convert to NumPy arrays
data = pd.read_csv("sonar.csv", header=None)
X = data.iloc[:, 0:60].values
y = data.iloc[:, 60].values

# encode class values as integers
encoder = LabelEncoder()
encoder.fit(y)
y = encoder.transform(y).reshape(-1, 1)

class SonarDataset(Dataset):
    def __init__(self, X, y):
        # convert into PyTorch tensors and remember them
        self.X = torch.tensor(X, dtype=torch.float32)
        self.y = torch.tensor(y, dtype=torch.float32)

    def __len__(self):
        # this should return the size of the dataset
        return len(self.X)

    def __getitem__(self, idx):
        # this should return one sample from the dataset
        features = self.X[idx]
        target = self.y[idx]
        return features, target

# set up DataLoader for data set
dataset = SonarDataset(X, y)
trainset, testset = random_split(dataset, [0.7, 0.3])
loader = DataLoader(trainset, shuffle=True, batch_size=16)

# create model
model = nn.Sequential(
    nn.Linear(60, 60),
    nn.ReLU(),
    nn.Linear(60, 30),
    nn.ReLU(),
    nn.Linear(30, 1),
    nn.Sigmoid()
)

# Train the model
n_epochs = 200
loss_fn = nn.BCELoss()
optimizer = optim.SGD(model.parameters(), lr=0.1)
model.train()
for epoch in range(n_epochs):
    for X_batch, y_batch in loader:
        y_pred = model(X_batch)
        loss = loss_fn(y_pred, y_batch)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
```

```
# create one test tensor from the testset
X_test, y_test = default_collate(testset)
model.eval()
y_pred = model(X_test)
acc = (y_pred.round() == y_test).float().mean()
acc = float(acc)
print("Model accuracy: %.2f%%" % (acc*100))
```

Listing 18.6: Complete example of training a model with `DataLoader` and `Dataset`

It is very similar to the example you have before. Beware that the PyTorch model still needs a tensor as input, not a `Dataset`. Hence in the above, you need to use the `default_collate()` function to collect samples from a dataset into tensors.

18.4 Further Readings

This section provides more resources on the topic if you are looking to go deeper.

Articles

torch.utils.data. PyTorch documentation.
https://pytorch.org/docs/stable/data.html

Datasets & DataLoaders. PyTorch tutorial.
https://pytorch.org/tutorials/beginner/basics/data_tutorial.html

18.5 Summary

In this chapter, you learned how to use `DataLoader` to create shuffled batches of data and how to use `Dataset` to provide data samples. Specifically you learned:

- `DataLoader` as a convenient way of providing batches of data to the training loop
- How to use `Dataset` to produce data samples
- How combine `Dataset` and `DataLoader` to generate batches of data on the fly for model training

As you have seen in many examples so far, you make use of scikit-learn for various tasks such as train-test split. In the next chapter, you will learn how to wrap your PyTorch model into a scikit-learn model so you can make use of its model selection routines.